DeepAutonomy

ScienceCUA: Resource-to-Skill Construction and Execution-Guided Skill Evolution for Scientific Computer Use Agents

Daiqiang Li[1], Jingyi Peng[1], Rui Hu[2], Jia Li[3], Xuyang Liu[4], Guanren Qiao[1], Yujia Xiang[1], Qiuzhuang Sun[5], Zhiping Xu[1], Mingrui Luo[1,†]

[1]DeepAutonomy Technology Co., Ltd. [2]Peking University [3]College of AI, Tsinghua University
[4]The Hong Kong Polytechnic University [5]Singapore Management University

Project Page Code Benchmark Environment

Abstract

Computer-use agents for scientific software require application-specific expertise. Much of this expertise is already documented in human-authored software manuals and web pages, making them readily available resources for improving agents' operational capabilities. However, turning these resources into executable, reusable skills remains challenging due to mismatches between documented and deployed software versions and differences in scope between documentation coverage and task requirements. To address these challenges, we introduce ScienceCUA, a two-stage framework combining resource-to-skill construction with execution-guided skill evolution. First, ScienceCUA identifies tasks supported by the software from its documentation, infers operational trajectories, and generalizes them into reusable Skills. Command catalogs, syntax constraints, and shortcuts are organized separately as References for on-demand lookup. Then, ScienceCUA uses execution feedback to drive skill self-evolution by contrasting successful and failed trajectories for the same task, identifying deficiencies, revising existing skills or adding new ones, and updating the skill library only after repeated validation in real environments. To enable reliable evaluation without benchmark noise or task leakage, we introduce two improved benchmarks. ScienceBoard-Verified provides an audited and corrected basis for measuring task performance, while ScienceBoard-VE separates skill evolution from final testing through disjoint evolution and test tasks and further supports native-agent evaluation. Experiments show that the initial Skills and References improve average task success rate by up to 7.23 percentage points over the base model, while skill self-evolution increases the total gain over the baseline to as much as 17.12 percentage points. ScienceCUA turns software documentation into an evolving set of executable skills and queryable references, offering a practical way for general-purpose agents to use scientific software reliably and at scale.

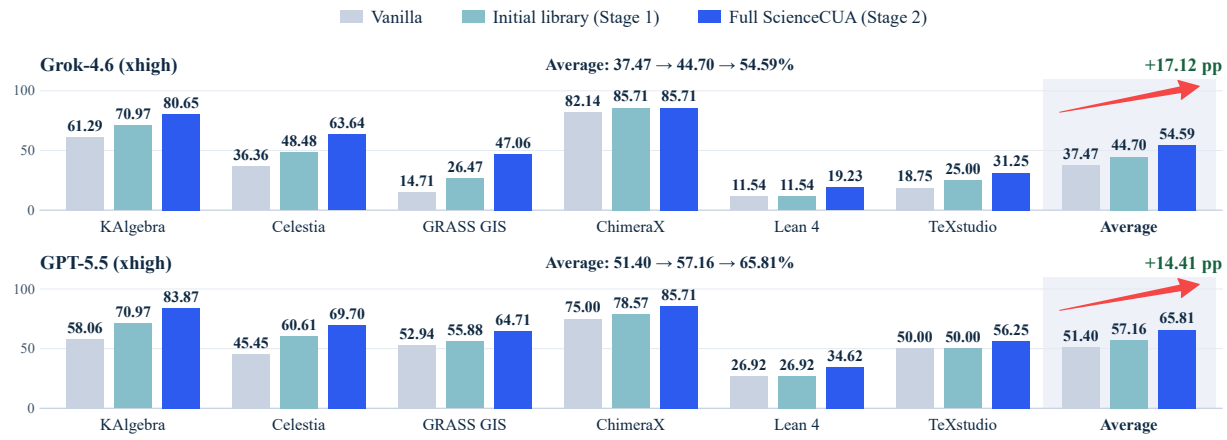


Figure 1: Two-stage gains on ScienceBoard-Verified. Success rates (%) for Vanilla, the initial library after Resource-to-Skill construction, and Full ScienceCUA after skill self-evolution.

[†]Corresponding author

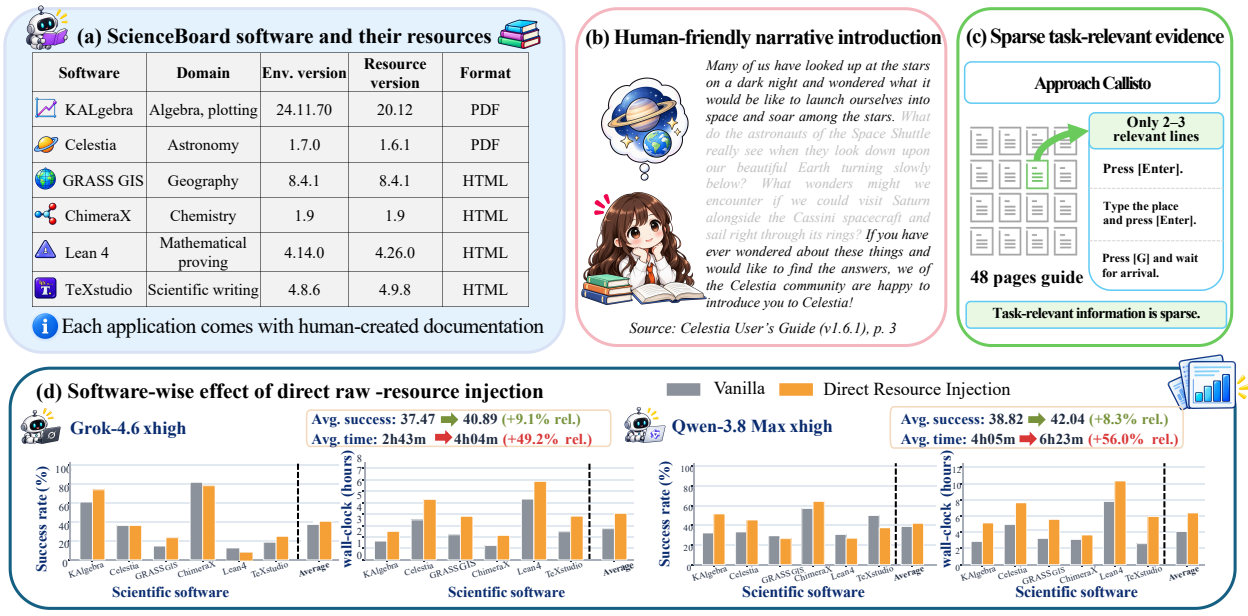


Figure 2: Motivation for resource-to-skill construction. (a) Scientific applications and their resources. (b) Human-oriented narrative content. (c) Sparse task-relevant information. (d) Effects of directly injecting raw resources on success rate and total execution time.

1 Introduction

Computer-use agents (CUAs) built on foundation models can now execute multi-step tasks through graphical and command-line interfaces (Qin et al., 2025; Sager et al., 2026; Xie et al., 2024). Most existing benchmarks, including WebArena, OSWorld, and Windows Agent Arena, focus on general-purpose web, office, and desktop tasks, while scientific software has received comparatively less attention (Xie et al., 2024; Zhou et al., 2024; Bonatti et al., 2024; Sun et al., 2026). Scientific software is widely used for experiments, simulations, and data analysis, but operating these tools often requires application-specific workflows, commands, syntax, and parameter constraints (Sun et al., 2026; Ai et al., 2026). To complete scientific-software tasks, agents must use software-specific knowledge to select appropriate operations and organize them into effective workflows (Shao et al., 2026).

We study this problem using ScienceBoard (Sun et al., 2026), which evaluates CUAs on six scientific applications. Figure 2(a) shows that each application provides user manuals or official tutorials containing application-specific knowledge. However, Figure 2(d) shows that directly placing these resources in the model context yields only limited gains in task success while substantially increasing execution time. Figure 2(b) illustrates that human-authored documentation mixes operational instructions with conceptual explanations and other narrative content. A particular task, however, typically depends on only a small fraction of the available information, as shown in Figure 2(c). The central question is how to extract task-relevant information from these human-oriented resources and organize it as structured knowledge that the model can use effectively.

Prior work has constructed reusable skills from human-created resources for content creation, gaming, software operation, and related applications (Fan et al., 2026; Che et al., 2026; Jiang et al., 2026; Shen et al., 2026). For scientific software, these resources must supply two kinds of knowledge with different lifetimes: procedural guidance for a specific task, and shared software knowledge queried on demand. We therefore introduce ScienceCUA, which extracts task-oriented procedural knowledge and cross-task shared software knowledge from raw software resources, organizes them

as Skills and References, respectively, and combines them on demand during execution.

However, the initial Skills and References constructed from raw resources have inherent limitations. First, the available PDF or HTML resources may not match the version of the software deployed in the execution environment, since perfectly matched resources are not always available. Second, for some applications, the resource coverage is broader than the range of tasks being evaluated, while the task scope is unknown during construction. The resulting knowledge library therefore contains many entries that are irrelevant to the evaluation tasks, reducing the density of useful information. Third, some task requirements may fall outside the coverage of the resources themselves.

To address these limitations, ScienceCUA drives the continual evolution of Skills and References through execution feedback. Prior work has used task feedback and execution trajectories to build reflective memory, optimize external skills, or adapt agent behavior (Shinn et al., 2023; Zhao et al., 2024; Ni et al., 2026; Liu et al., 2026; Yang et al., 2026; Chen et al., 2026; Zhang et al., 2026; Sun et al., 2025). In contrast, ScienceCUA improves a scientific-software knowledge library initialized from software resources, using historical failure trajectories to guide subsequent exploration and help agents avoid repeating failed paths or continuing to follow potentially erroneous Skills and References; when knowledge is insufficient, it revisits the original resources to retrieve and construct task-relevant Skills or References. Combined with on-demand composition, this design both helps agents bypass ineffective paths caused by version mismatch and, where the original resources support it, supplements omitted or overly general knowledge in the initial library, thereby reducing the mismatch between the resources and the actual tasks. Candidate updates are added only after independent validation in the target environment.

To support reliable evaluation, we audit and repair ScienceBoard to create ScienceBoard-Verified. We also add a set of tasks that do not overlap with the final test tasks, use them exclusively for skill evolution, and combine the audited benchmark and the new tasks into ScienceBoard-VE; in addition, we extend the evaluation framework to support native agents. Across six scientific applications, the initial Skills and References improve the average task success rate over the base model by up to 7.23 percentage points. Execution-guided skill evolution increases the total gain over the baseline to as much as 17.12 percentage points. Our contributions are:

1. **Resource-to-skill construction for scientific software.** We present a method for constructing a knowledge library from raw software resources. It organizes task-oriented procedural knowledge as Skills and cross-task shared software knowledge as References, and supports their on-demand composition during execution.

2. **Execution-feedback-driven skill evolution.** We use failure trajectories to guide exploration and help agents avoid repeatedly following erroneous Skills or References. When knowledge is missing or an existing entry is inapplicable, the system supplements or revises Skills and References from the original resources and updates the knowledge library only after validation in the target environment.

3. **Reliable evaluation of skill construction and evolution.** We audit and repair ScienceBoard to create ScienceBoard-Verified, add skill-evolution tasks disjoint from the final test set to form ScienceBoard-VE, and extend evaluation to native agents.

2 Related Work

Resource-to-Skill Learning. Automatically constructing reusable agent knowledge from external resources is a relatively recent research direction. MMG2Skill Che et al. (2026) converts public

guides into editable Skills, but assumes that tasks are paired with relevant guides in advance and does not realize an end-to-end pipeline for constructing Skills directly from raw resources. VisualSkill Jiang et al. (2026) leverages document images to support GUI element identification and state verification. However, some HTML documentation contains no images and thus provides no additional visual evidence; even when screenshots are available, their interfaces may differ from the current software version, potentially introducing misleading visual cues. SkillFoundry Shen et al. (2026) supports workflow generation for concrete scientific tasks, but focuses on domain capability coverage and task workflows rather than on systematically organizing reusable knowledge for a given scientific software package. Overall, existing methods differ in their resource formats, task assumptions, and construction objectives, but systematic investigation is still lacking into how to automatically discover tasks from raw scientific-software resources and organize cross-task reusable knowledge without pre-specified task–resource pairs.

Self-Evolving Skills and Experience-Driven Agent Adaptation. Skill self-evolution improves reusable agent knowledge from execution experience without modifying model parameters. Early work such as Reflexion Shinn et al. (2023) used verbal feedback to improve subsequent behavior, and SEAgent Sun et al. (2025) extended this experience-driven paradigm to computer-use agents. Later work made such experience explicit and reusable as skills. EvoSkill Alzubi et al. (2026) creates or revises skills from failure feedback and selects candidates through validation, while Trace2Skill Ni et al. (2026) distills trajectory-local lessons into transferable skill catalogs. As skill updates become central, subsequent studies increasingly focus on their reliability. SkillOpt Yang et al. (2026) constrains the scope of skill edits and accepts only validation-improving updates, while SkillCAT Chen et al. (2026) contrasts successful and failed trajectories for the same task and uses task replay before merging candidate edits. Most prior methods start from execution experience or an existing skill library. ScienceCUA instead initializes its library from scientific-software documentation and addresses version and task-coverage mismatch through failure-history-guided exploration and source-grounded additions or revisions.

3 Method

3.1 Resource-to-Skill Construction

ScienceCUA first transforms software documentation into an initial library of reusable Skills and queryable References, and then evolves this library using execution feedback from a disjoint set of evolution tasks.

Decoupling Skills and References

Given PDF/HTML software documentation $\mathcal{D}$, we construct an initial library $\mathcal{L}_0 = (\mathcal{S}_0, \mathcal{R}_0)$, where Skills $\mathcal{S}_0$ encode **reusable task-level operational procedures** and References $\mathcal{R}_0$ organize **cross-task shared knowledge**, such as commands, syntax, and parameter constraints. For example, KAlgebra's 2D and 3D plotting workflows form separate Skills, while syntax for square roots and exponentiation is stored in a Reference (Figure 4). During execution, the agent selects a Skill for the required workflow and consults References as needed to incorporate software knowledge into its actions.

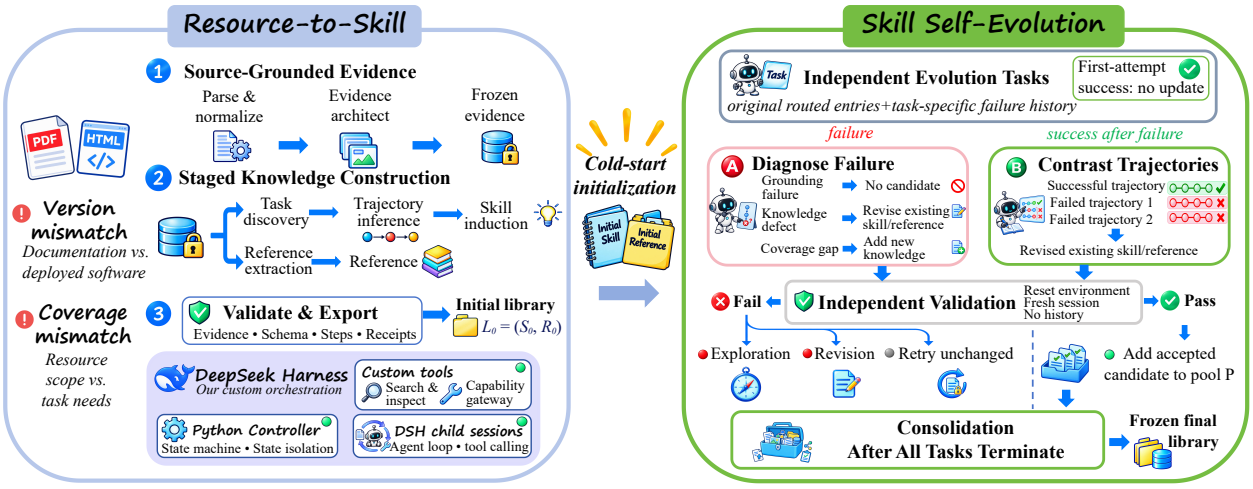


Figure 3: ScienceCUA overview. Resource-to-Skill construction builds Skills and References from PDF/HTML documentation to initialize execution-guided Skill self-evolution.

Construction Pipeline

We represent a Skill as $s = (n_s, d_s, a_s, p_s, b_s)$, where n_s is its name, d_s is its capability description, a_s specifies its applicability, p_s lists its preconditions, and b_s contains its procedural body, including the steps needed to accomplish the goal. We represent a Reference as $r = (n_r, d_r, c_r)$, where n_r is its name, d_r describes its purpose, and c_r contains the concrete lookup content. Construction first organizes the documentation into a shared evidence corpus. From this corpus, one branch extracts References, while the other discovers Tasks, infers trajectories, and induces Skills.

Traceable evidence organization. To support all downstream stages and reduce hallucinations, we organize documentation into traceable evidence. A parser uses authored headings and navigation to structure text and visual content while retaining source locations. An agent refines this organization without rewriting the content. Every downstream Reference, Task, and inferred action or observation must be grounded in this evidence.

Reference extraction. To make shared software knowledge accessible during execution, the Reference branch turns relevant evidence into focused lookup entries. A program first extracts and deduplicates candidate lookup content from the documentation, including command tables, syntax constraints, and shortcut descriptions. An agent reviews the remaining candidates to decide what to retain and how to group them, preserving documented syntax, parameters, and examples.

Task discovery, trajectory inference, and Skill induction. To ground reusable Skills in concrete operational requirements, the other branch proceeds from goals to trajectories before induction. An agent first discovers source-supported user goals without benchmark task input. For each Task, it infers ordered actions and expected observations, retaining documented controls. These trajectories make the required operations explicit before related procedures are grouped into a coherent Skill, helping preserve actionable detail during abstraction. After induction, we check whether the operational capability expressed by each trajectory is included in a Skill and explicitly record any exclusions.

Each stage produces structured outputs that serve as inputs to subsequent stages, yielding the initial library $\mathcal{L}_0 = (\mathcal{S}_0, \mathcal{R}_0)$. Reference extraction, Task discovery, trajectory inference, and Skill induction are assigned to independent agents, each with its own tools, prompt, and required output format. A unified description of the Resource-to-Skill pipeline, including agent roles and tools, prompts, and output schemas, is provided in Appendix F.

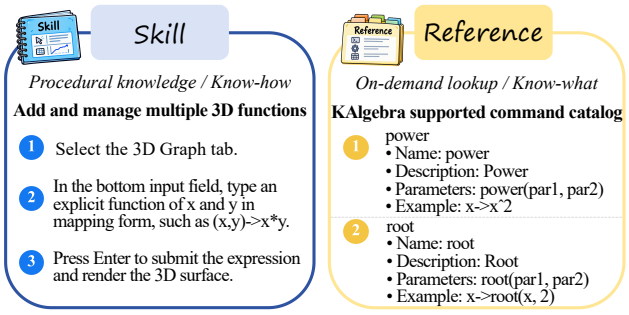


Figure 4: An example in KAlgebra: a procedural Skill for plotting 3D functions (left) and a Reference cataloging supported commands (right).

3.2 Execution-Guided Skill Self-Evolution

Differences between documented and deployed software versions, together with gaps between resource coverage and task requirements, can leave the initial library misaligned with the target environment or incomplete for its tasks. ScienceCUA therefore keeps the execution model fixed and continually evolves Skills and References through two mechanisms: **failure-history-guided exploration** and **source-grounded knowledge addition**.

Evolution setting and overall workflow. As described in Section 4.2, ScienceBoard-VE contains an evolution set $E = \text{Expanded} \setminus \text{Verified}$ and a held-out test set $V = \text{Verified}$, where $E \cap V = \emptyset$. Evolution uses E, while V remains unseen until final evaluation. Given the initial library $\mathcal{L}_0 = (\mathcal{S}_0, \mathcal{R}_0)$, ScienceCUA processes each task $q \in E$ independently and maintains its history $\mathcal{H}_q$. Histories and provisional updates are not shared across tasks.

Each task uses two types of execution. **Task exploration** attempts the task with the frozen initial library, together with any prior failure records available, to search for a feasible path and collect environmental feedback. **Candidate validation** temporarily loads one modified or newly added Skill or Reference and reruns the task to test whether that candidate can independently support successful execution.

Each task begins with task exploration. A first-exploration success requires no update and exits the evolution process. After a failure, the system either makes no change, revises existing knowledge, or adds source-supported knowledge according to its diagnosis. Any modified or newly added Skill or Reference undergoes candidate validation. A successful candidate enters the candidate pool $\mathcal{P}$. A failed candidate and its validation trajectory enter $\mathcal{H}_q$ to guide the next task exploration. If a later exploration succeeds with the help of failure history, the system contrasts it with earlier failures, derives a revision candidate when needed, and validates it. Each task contributes at most one validated candidate to $\mathcal{P}$. These candidates are then consolidated into $\mathcal{L}_{\text{final}}$, merged targets undergo regression certification, and the library is frozen for evaluation. Further details are provided in Appendix I.

Failure-history-guided exploration. The first exploration of task q performs normal routing over $\mathcal{L}_0$ by selecting entries from their names and descriptions; the routed Skills and References define $\mathcal{F}_q \subseteq \mathcal{L}_0$. Later task explorations load the same entries in their initial versions and additionally receive the accumulated failure history. Formally, the i-th exploration is $\tau_q^{(i)} = \text{Execute}_\theta(q; \mathcal{L}_0, \mathcal{F}_q, \mathcal{H}_q^{(i)})$, where $\mathcal{H}_q^{(1)} = \emptyset$. For $i > 1$, $\mathcal{H}_q^{(i)}$ contains the task's earlier failed task explorations and candidate validations, including their trajectories, screenshots, and outcomes. These failure records help the model avoid paths that failed because of version mismatch, rather than

mechanically follow initial knowledge that conflicts with the deployed software. Later explorations always start from the same $\mathcal{L}_0$ and $\mathcal{F}_q$: failure history changes exploration behavior, whereas knowledge changes still require independent validation. Appendix I.2 specifies the archived evidence and its input format.

Failure diagnosis and candidate construction. For a failed task exploration, the diagnosis module analyzes its loaded Skills and References, trajectory, screenshots, and evaluator outcome, and assigns one primary cause: (1) **grounding failure**, where interface perception or manipulation is the main obstacle and no candidate is produced; (2) **modify existing knowledge**, where one entry in $\mathcal{F}_q$ is relevant but contains an incorrect or incomplete procedure, condition, parameter, or version-dependent instruction; or (3) **add new knowledge**, where the routed knowledge has a genuine coverage gap and triggers retrieval from the original resources $\mathcal{D}$. The modify branch produces one complete revision candidate $\Delta_q^{(i)}$; the add branch produces a new Skill or Reference candidate only when retrieval finds sufficient source support. Any resulting candidate is independently validated: success adds it to $\mathcal{P}$ and terminates task q, whereas failure records it in $\mathcal{H}_q$ before task exploration resumes. Separately, only when a later task exploration itself succeeds does the system contrast that success with earlier failures and either revise one entry in $\mathcal{F}_q$ or make no change; this branch cannot add knowledge. Details are provided in Appendices I.3 and I.4.

Independent candidate validation. For candidate $\Delta_q^{(i)}$, ScienceCUA constructs the temporary library $\widetilde{\mathcal{L}}_q^{(i)} = \mathcal{L}_0 \oplus \Delta_q^{(i)}$, where $\oplus$ replaces one initial entry with its revision or inserts one source-supported new entry. Validation then executes $\widetilde{\tau}_q^{(i)} = \mathrm{Execute}_\theta(q; \widetilde{\mathcal{L}}_q^{(i)}, \widetilde{\mathcal{F}}_q^{(i)}, \emptyset)$ in a reset environment and a fresh session. It fixes the route obtained by the first task exploration, with the candidate entry substituted or added, and receives no prior trajectory, diagnosis, or candidate rationale, thereby testing whether the candidate knowledge itself supports the task. A successful candidate enters $\mathcal{P}$; a failed candidate remains inactive, and its execution record is added to the task's failure history. Appendix I.3 gives the complete validation protocol.

4 ScienceBoard-VE: A Verified and Expanded Benchmark for Scientific Computer-Use Agents

ScienceBoard (Sun et al., 2026) is a benchmark for scientific computer-use agents and has been adopted in subsequent studies on agent evaluation, training, and task expansion (Lv et al., 2026; Lu et al., 2026). During reproduction, we audited all 169 original tasks and identified issues in task instructions, task statements, reference answers, evaluators, and execution environments. We therefore construct **ScienceBoard-Verified** and add a disjoint set of tasks for skill self-evolution. Together, they form **ScienceBoard-VE**. We also extend the evaluation framework to support native-agent evaluation. Figure 5 summarizes the task distribution and benchmark quality assurance.

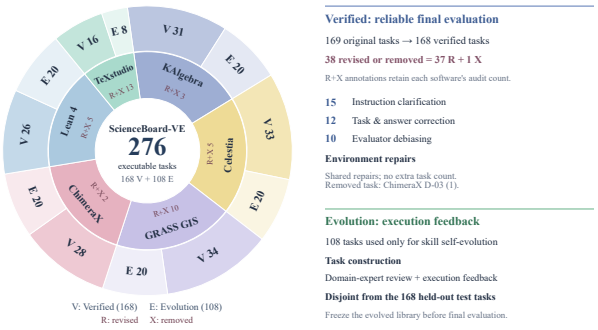


Figure 5: Task distribution in ScienceBoard-VE. The inner ring groups tasks by application. The outer ring separates verified evaluation tasks (V) from evolution tasks (E). R+X denotes revised or removed tasks in the original benchmark. The removed task is excluded from the sector sizes.

4.1 ScienceBoard-Verified

We substantively revise 37 tasks and remove one
additional task, yielding 168 executable tasks reserved for final evaluation. We organize the revisions and supporting repairs into four categories: (1) **Instruction Ambiguity**, which covers 15 revised tasks with unclear goals or omitted completion requirements; (2) **Task & Answer Errors**, which covers 12 revised tasks involving incorrect task statements, reference answers, or expected evaluator results; (3) **Evaluator Bias**, which covers 10 revised tasks in which unnecessary restrictions exclude valid execution paths, output representations, or output locations; and (4) **Environment Repairs**, which covers application prompts, response handling, runtime and initialization settings, and unavailable external dependencies. The fourth category includes shared evaluation-environment repairs and the removed task; these do not add to the 37 revised tasks. Appendix G details all 37 task revisions, the task removal rationale, and the associated evaluation-environment repairs.

4.2 Expanding the Benchmark for Skill Self-Evolution

To separate skill improvement from final evaluation, we add 108 tasks that do not overlap with the verified test set: 20 each for KAlgebra, Celestia, GRASS GIS, ChimeraX, and Lean 4, and 8 for TeXstudio. These tasks are used exclusively for **skill self-evolution**, while the 168 tasks in ScienceBoard-Verified remain unseen during skill construction and evolution and are used only for final evaluation. The resulting **ScienceBoard-VE** contains 276 executable tasks in total. The evolution tasks use the same types of software operations as the test tasks but differ in their specific targets or requirements, allowing execution experience to transfer without reusing the test tasks themselves. See Appendix H for details.

4.3 Native-Agent Evaluation

The original ScienceBoard evaluates models through a unified API-based interaction loop. We extend it to support native evaluation of Codex and Grok Build, keeping tasks, software environments, and evaluators unchanged. In both settings, ScienceBoard supplies screenshots and accessibility trees, executes the returned action code or control instructions, and evaluates task outcomes. In the model-based setting, each API request includes the system prompt, task instruction, screenshots and accessibility trees from the three most recent steps together with the model-generated actions for those steps, and the current screenshot and accessibility tree. For ScienceCUA, we implement two-stage resource routing through progressive disclosure. Before execution, the evaluated model selects relevant Skills and References from their names and descriptions based on the task instruction. The selected entries are then loaded in full into the system prompt and included in every subsequent execution request. In the native-agent setting, each task uses a separate persistent session, to which the outer evaluation framework appends only the current screenshot and accessibility tree at each step. The native harness maintains the task history and compacts context as needed. Skills and References are registered as independently loadable native resources, which the agent selects and reads during execution. Each turn concludes with one ScienceBoard action block for the evaluation framework to execute. To prevent agents from obtaining additional reference material through web search, we disable web search for native agents. To preserve task independence, conversation history and runtime context are maintained separately for each task and are not shared between tasks.

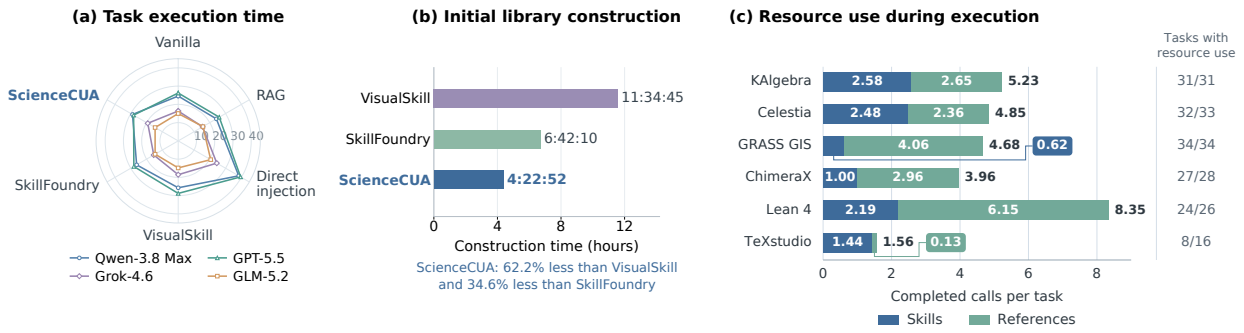


Figure 6: Task execution time, initial library construction cost, and resource use during execution. (a) Total task execution time summed across all six applications for four models under six methods. Each curve represents one model, and radial values indicate hours. (b) Total initial library construction time across the six applications for VisualSkill, SkillFoundry, and ScienceCUA, using GPT-5.5 (xhigh). (c) Completed calls to Skills and References in final task attempts, averaged over all tasks in each application, for native Grok Build with Grok-4.6 (xhigh). White labels give Skill and Reference calls per task in their respective colors; the narrow GRASS GIS Skill and TeXstudio Reference segments have connected labels. Bar-end labels give the combined calls per task. Values are rounded independently. The fractions on the right show the number of tasks with at least one resource call over the total number of tasks in each application.

5 Experiments

Experimental setup. We evaluate ScienceCUA on ScienceBoard-Verified. Model-based evaluation uses Qwen-3.8 Max, GLM-5.2, Grok-4.6, and GPT-5.5. Native-agent evaluation uses Grok Build with Grok-4.6 and Codex with GPT-5.5. The two settings follow the respective interaction and resource access protocols described in Section 4.3. In model-based ScienceCUA evaluation, at most three Skill and Reference entries are selected in total before each task. In native-agent evaluation, the agent autonomously selects and reads resources during execution, with no limit on the number of Skill and Reference entries it can access. We report task success rates for each of the six applications and use their unweighted mean as the overall performance metric.

Model-based evaluation compares ScienceCUA with Vanilla, direct document injection, RAG, VisualSkill, and SkillFoundry. Native-agent evaluation compares Vanilla, VisualSkill, SkillFoundry, and ScienceCUA. ScienceCUA and VisualSkill use the same documentation inputs. SkillFoundry does not support direct documentation input and instead automatically searches for and collects online resources based on the name and version of the software installed in the execution environment. Implementation details are provided in Appendix B for direct injection, Appendix C for RAG, Appendix D for SkillFoundry, and Appendix E for VisualSkill.

Skill self-evolution uses the 108 additional tasks in ScienceBoard-VE. After evolution, the knowledge library is frozen and evaluated on ScienceBoard-Verified without access to evolution trajectories or histories.

5.1 RQ1: Can ScienceCUA-constructed resources improve task success in scientific software operation?

Table 1 shows that the initial ScienceCUA library achieves the highest average success rate in all four model-based settings. ScienceCUA improves over Vanilla by 5.76–7.23 percentage points, compared with 1.53–3.42 points for direct document injection, and outperforms RAG, VisualSkill, and SkillFoundry across all four models. The gains also extend to native-agent evaluation. ScienceCUA reaches average success rates of 55.41% with Grok Build and 59.82% with Codex, improving over their respective Vanilla baselines by 9.12 and 5.73 percentage points and outperforming VisualSkill and SkillFoundry in both settings. Figure 6(a) shows that ScienceCUA requires 21.1–28.9% less

Table 1: Success rates on ScienceBoard-Verified under model-based (top) and native-agent (bottom) evaluation. Bold marks column-wise best results within each group.

Model	Method	KAlgebra	Celestia	GRASS GIS	ChimeraX	Lean 4	TeXstudio	Average	
Qwen-3.8 Max (xhigh)	Vanilla	32.26	33.33	29.41	57.14	30.77	50.00	38.82	
	RAG	38.71	36.36	**32.35**	53.57	30.77	50.00	40.29	(↑ 3.79%)
	Direct injection	**51.61**	**45.45**	26.47	**64.29**	26.92	37.50	42.04	(↑ 8.29%)
	visualskill	38.71	33.33	**32.35**	57.14	30.77	50.00	40.38	(↑ 4.02%)
	skillfoundry	35.48	36.36	20.59	53.57	**34.62**	50.00	38.44	(↓ 0.98%)
	ScienceCUA	**51.61**	36.36	**32.35**	**64.29**	30.77	**56.25**	**45.27**	(↑ 16.62%)
GLM-5.2 (Max)	Vanilla	19.35	12.12	0.00	53.57	3.85	6.25	15.86	
	RAG	22.58	**18.18**	0.00	60.71	3.85	**12.50**	19.64	(↑ 23.83%)
	Direct injection	25.81	15.15	0.00	57.14	0.00	6.25	17.39	(↑ 9.65%)
	visualskill	**29.03**	9.09	0.00	**64.29**	**7.69**	6.25	19.39	(↑ 22.26%)
	skillfoundry	19.35	12.12	**2.94**	50.00	3.85	**12.50**	16.79	(↑ 5.86%)
	ScienceCUA	**29.03**	15.15	**2.94**	**64.29**	**7.69**	**12.50**	**21.93**	(↑ 38.27%)
Grok-4.6 (xhigh)	Vanilla	61.29	36.36	14.71	82.14	**11.54**	18.75	37.47	
	RAG	70.97	36.36	20.59	82.14	7.69	**31.25**	41.50	(↑ 10.76%)
	Direct injection	**74.19**	36.36	23.53	78.57	7.69	25.00	40.89	(↑ 9.13%)
	visualskill	61.29	36.36	23.53	78.57	**11.54**	**31.25**	40.42	(↑ 7.87%)
	skillfoundry	45.16	33.33	20.59	**85.71**	**11.54**	**31.25**	37.93	(↑ 1.23%)
	ScienceCUA	70.97	**48.48**	**26.47**	**85.71**	**11.54**	25.00	**44.70**	(↑ 19.30%)
GPT-5.5 (xhigh)	Vanilla	58.06	45.45	52.94	75.00	26.92	**50.00**	51.40	
	RAG	64.52	54.55	50.00	**78.57**	**30.77**	50.00	54.73	(↑ 6.48%)
	Direct injection	67.74	54.55	**55.88**	75.00	23.08	43.75	53.33	(↑ 3.75%)
	visualskill	64.52	57.58	52.94	75.00	26.92	**50.00**	54.49	(↑ 6.01%)
	skillfoundry	61.29	54.55	50.00	**78.57**	26.92	**50.00**	53.56	(↑ 4.20%)
	ScienceCUA	**70.97**	**60.61**	**55.88**	**78.57**	26.92	**50.00**	**57.16**	(↑ 11.21%)
Agent	**Method**	**KAlgebra**	**Celestia**	**GRASS GIS**	**ChimeraX**	**Lean 4**	**TeXstudio**	**Average**	
Grok Build Grok-4.6 (xhigh)	Vanilla	70.97	42.42	35.29	78.57	19.23	31.25	46.29	
	visualskill	70.97	63.64	35.29	75.00	26.92	31.25	50.51	(↑ 9.12%)
	skillfoundry	64.52	51.52	26.47	78.57	**30.77**	**43.75**	49.27	(↑ 6.44%)
	ScienceCUA	**77.42**	**66.67**	**38.24**	**85.71**	26.92	37.50	**55.41**	(↑ 19.70%)
GPT-5.5 (xhigh)+Codex	Vanilla	58.06	51.52	52.94	75.00	**30.77**	**56.25**	54.09	
	visualskill	61.29	57.58	55.88	**78.57**	26.92	**56.25**	56.08	(↑ 3.68%)
	skillfoundry	61.29	54.55	55.88	75.00	**30.77**	50.00	54.58	(↑ 0.91%)
	ScienceCUA	**70.97**	**60.61**	**61.76**	**78.57**	**30.77**	**56.25**	**59.82**	(↑ 10.59%)

total task execution time than direct document injection across the four models. ScienceCUA also reduces total initial library construction time by 62.2% and 34.6% relative to VisualSkill and SkillFoundry, respectively (Figure 6(b)). These results support the usefulness of the constructed Skills and References as initial operational knowledge for scientific software.

5.2 RQ2: Do Skills and References make complementary contributions to task success?

We ablate Skills and References separately to assess the individual and joint contributions of procedural guidance and cross-task shared lookup knowledge to task success. Skills organize complete operational procedures around specific goals, while References provide shared knowledge that is not tied to any single procedure, including syntax constraints, parameter rules, command and function catalogs, and keyboard shortcuts. The two resource types have no one-to-one correspondence that permits direct, equivalent merging. We therefore do not use a forced merger of the two resource types as an information-matched control for knowledge organization, and instead use resource ablations to evaluate the value of providing each knowledge type individually and both jointly.

Table 2 shows that providing either resource type alone improves average success rates over Vanilla, while providing both achieves the best results. In model-based evaluation with Grok-4.6,

Table 2: Ablation of skills and references on ScienceBoard-Verified.

Model	Method	KAlgebra	Celestia	GRASS GIS	ChimeraX	Lean 4	TeXstudio	Average	
Grok-4.6 (xhigh)	Vanilla	61.29	36.36	14.71	82.14	**11.54**	18.75	37.47	
	skill-only	67.74	39.39	14.71	82.14	**11.54**	**25.00**	40.09	(↑ 6.99%)
	reference-only	64.52	36.36	20.59	**85.71**	**11.54**	**25.00**	40.62	(↑ 8.41%)
	skill+reference	**70.97**	**48.48**	**26.47**	**85.71**	**11.54**	**25.00**	**44.70**	(↑ 19.30%)
Grok Build Grok-4.6 (xhigh)	Vanilla	70.97	42.42	35.29	78.57	19.23	31.25	46.29	
	skill-only	74.19	54.55	35.29	78.57	19.23	**37.50**	49.89	(↑ 7.78%)
	reference-only	**77.42**	48.48	**38.24**	82.14	23.08	**37.50**	51.14	(↑ 10.48%)
	skill+reference	**77.42**	**66.67**	**38.24**	**85.71**	**26.92**	**37.50**	**55.41**	(↑ 19.70%)

the average success rates are 40.09% with Skills alone, 40.62% with References alone, and 44.70% with both. The corresponding results for native Grok Build are 49.89%, 51.14%, and 55.41%. The relative gains from the two resource types vary across applications, while providing both further improves overall performance, supporting the complementary contributions of procedural guidance and shared lookup knowledge to task execution.

The KAlgebra C-05 comparison in Appendix J illustrates these roles under the same budget of five interaction turns. Each condition retains autonomous routing over the permitted resource types. Reference-only spends its budget locating the script-loading interface; Skill-only loads the script but leaves the calculation symbolic. With both types, the agent combines the script-loading Skill's guidance with application-specific knowledge from the function-catalog Reference, then corrects an intermediate vector-valued result using execution feedback to complete the task.

Figure 6(c) further illustrates actual calls to both resource types during task execution. Native Grok Build with Grok-4.6 (xhigh) accesses at least one resource in 156 of the 168 tasks (92.9%). Final task attempts contain 291 completed Skill calls and 543 completed Reference calls, with repeated calls to the same entry counted separately. These records show that the agent calls both Skills and References during actual task execution and, together with the preceding ablation results, support the contributions of both knowledge types to task completion.

5.3 RQ3: Can self-evolution mitigate the limitations of Resource-to-Skill construction?

The initial library may lack task-relevant knowledge or contain instructions incompatible with the deployed software version. We use Grok-4.6 (xhigh) and GPT-5.5 (xhigh) to compare the initial library, Full ScienceCUA, two ablated variants, Trace2Skill (Ni et al., 2026), and SkillCAT (Chen et al., 2026), examining whether execution-guided self-evolution improves performance on unseen tasks. The −**Knowledge addition** variant only revises existing entries without adding new Skills or References. The −**Failure history** variant withholds prior exploration and candidate-validation records from subsequent exploration, leaving the other mechanisms unchanged. Within each model setting, all evolution variants use the same initial library, evolution tasks, software environments, and runtime configurations. Detailed ScienceCUA settings are provided in Appendix I.

Table 3 shows that self-evolution improves the average success rate from 44.70% to 54.59% for Grok-4.6 and from 57.16% to 65.81% for GPT-5.5, yielding gains of 9.89 and 8.65 percentage points, respectively. Full ScienceCUA also outperforms Trace2Skill and SkillCAT, which achieve average success rates of 46.44% and 53.02% with Grok-4.6, and 60.46% and 63.74% with GPT-5.5, respectively. These results indicate that knowledge improvements obtained during evolution transfer to final test tasks not used during evolution.

Removing either knowledge addition or failure history reduces average success rates, but the dominant source of improvement varies by application. KAlgebra and Celestia are particularly sensitive to failure history: removing it reduces success by 9.68 percentage points for KAlgebra under both models, and by 9.09 and 6.06 points for Celestia under Grok-4.6 and GPT-5.5, respec-

Table 3: Self-evolution comparison and ablation on ScienceBoard-Verified. Results are success rates in percent. All variants are evaluated with frozen libraries and no evolution history.

Model	Method	KAlgebra	Celestia	GRASS GIS	ChimeraX	Lean 4	TeXstudio	Average	
Grok-4.6 (xhigh)	Full ScienceCUA	80.65%	63.64%	47.06%	85.71%	19.23%	31.25%	**54.59%**	
	−Knowledge addition	80.65%	63.64%	32.35%	85.71%	15.38%	31.25%	51.50%	(↓ 5.66%)
	−Failure history	70.97%	54.55%	41.18%	82.14%	15.38%	25.00%	48.20%	(↓ 11.71%)
	initial library	70.97%	48.48%	26.47%	85.71%	11.54%	25.00%	44.70%	(↓ 18.12%)
	Trace2Skill	70.97%	51.52%	23.53%	82.14%	19.23%	31.25%	46.44%	
	SkillCAT	74.19%	63.64%	44.12%	85.71%	19.23%	31.25%	53.02%	
GPT-5.5 (xhigh)	Full ScienceCUA	83.87%	69.70%	64.71%	85.71%	34.62%	56.25%	**65.81%**	
	−Knowledge addition	80.65%	69.70%	58.82%	82.14%	30.77%	56.25%	63.05%	(↓ 4.19%)
	−Failure history	74.19%	63.64%	61.76%	82.14%	30.77%	50.00%	60.42%	(↓ 8.19%)
	initial library	70.97%	60.61%	55.88%	78.57%	26.92%	50.00%	57.16%	(↓ 13.14%)
	Trace2Skill	74.19%	60.61%	58.82%	82.14%	30.77%	56.25%	60.46%	
	SkillCAT	77.42%	66.67%	61.76%	85.71%	34.62%	56.25%	63.74%	

tively. Documentation-version mismatches leave some initial entries with inapplicable capability constraints or interface paths. Appendix K traces how execution feedback repairs such a constraint in KAlgebra, while Appendix M shows how outdated interface guidance consumes execution steps in an initial Celestia run. GRASS GIS benefits especially from knowledge addition: removing it reduces success by 14.71 and 5.89 points under the two models. Its broad documentation leaves task-relevant module options underspecified in the initial library; Appendix L shows how adding a module-level Reference fills this gap and supports related Verified tasks. Gains are smaller or less consistent for the other applications. Lean 4 tasks require specialized formal proofs that the general theorem-proving guide rarely supplies directly, while many TeXstudio tasks require information from other software beyond the TeXstudio manual. Appendix N compares these task requirements with the source documentation.

6 Conclusion

We present ScienceCUA, a two-stage framework for computer-use agents operating scientific software. The first stage extracts source-grounded operational knowledge from software documentation, distilling task-level workflows into executable Skills and organizing commands, syntax, and parameter constraints as queryable References. The second stage refines Skills and References using execution feedback from the target software environment. We also construct the audited ScienceBoard-Verified and add evolution tasks disjoint from the test set to form ScienceBoard-VE, supporting knowledge evolution and native-agent evaluation. Experiments across six scientific applications show that the initial Skills and References improve average task success by up to 7.23 percentage points over the baseline in model-based evaluation. Knowledge self-evolution increases the total gain to as much as 17.12 percentage points. These results suggest that human-authored software resources can be transformed into executable, verifiable operational knowledge that continually improves through environmental feedback, enhancing general-purpose agents' ability to use scientific software. By maintaining Skills and References outside the model parameters, ScienceCUA offers a practical path for agents to adapt to changes in scientific software environments and task requirements without retraining the model.

References

Kuangshi Ai, Haichao Miao, Kaiyuan Tang, Nathaniel Gorski, Jianxin Sun, Guoxi Liu, Helgi I. Ingolfsson, David Lenz, Hanqi Guo, Hongfeng Yu, Teja Leburu, Michael Molash, Bei Wang, Tom Peterka, Chaoli Wang, and Shusen Liu. SciVisAgentBench: A Benchmark for Evaluating Scientific Data Analysis and Visualization Agents. *arXiv preprint arXiv:2603.29139*, 2026. URL https://arxiv.org/abs/2603.29139.

Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026.

Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, Lawrence Jang, and Zack Hui. Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale. *arXiv preprint arXiv:2409.08264*, 2024. URL https://arxiv.org/abs/2409.08264.

Xinyu Che, Junqi Xiong, Yunfei Ge, Xinping Lei, Shihao Li, Hang Yan, Han Li, Yuanxing Zhang, Zhiqi Bai, Jinhua Hao, Ming Sun, Han Li, and Jiaheng Liu. MMG2Skill: Can Agents Distill In-the-Wild Guides into Self-Evolving Skills? *arXiv preprint arXiv:2606.01993*, 2026. URL https://arxiv.org/abs/2606.01993.

Kunfeng Chen, Qihuang Zhong, Juhua Liu, and Bo Du. SkillCAT: Contrastive Assessment and Topology-Aware Skill Self-Evolution for LLM Agents. *arXiv preprint arXiv:2606.13317*, 2026. URL https://arxiv.org/abs/2606.13317v1.

Yijia Fan, Zonglin Di, Zimo Wen, Yifan Yang, Mingxi Cheng, Qi Dai, Bei Liu, Kai Qiu, Yue Dong, Ji Li, and Chong Luo. RESOURCE2SKILL: Distilling Executable Agent Skills from Human-Created Multimodal Resources. *arXiv preprint arXiv:2606.29538*, 2026. URL https://arxiv.org/abs/2606.29538.

Ziyan Jiang, Li An, Yujian Liu, Jiabao Ji, Qiucheng Wu, Jacob Andreas, Yang Zhang, and Shiyu Chang. VISUALSKILL: Multimodal Skills for Computer-Use Agents. *arXiv preprint arXiv:2606.18448*, 2026. URL https://arxiv.org/abs/2606.18448.

Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. SkillForge: Forging Domain-Specific, Self-Evolving Agent Skills in Cloud Technical Support. *arXiv preprint arXiv:2604.08618*, 2026. URL https://arxiv.org/abs/2604.08618.

Dunjie Lu, Shuai Bai, Tianyi Bai, Sicheng Fan, Chang Gao, Jian Guan, Feng Hu, Mianqiu Huang, Xingyang Huang, Yizhen Jiang, et al. Qwen-cua: Native computer use for (almost) everything. *arXiv preprint arXiv:2608.02352*, 2026.

Bowen Lv, Xiao Liu, Yanyu Ren, Hanyu Lai, Bohao Jing, Hanchen Zhang, Yanxiao Zhao, Shuntian Yao, Jie Tang, and Yuxiao Dong. Scalecua: Scaling computer use agents with verifiable task synthesis and efficient online rl. *arXiv preprint arXiv:2607.11185*, 2026.

Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2Skill: Distill Trajectory-Local Lessons into Transferable Agent Skills. *arXiv preprint arXiv:2603.25158*, 2026. URL https://arxiv.org/abs/2603.25158.

Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. *arXiv preprint arXiv:2501.12326*, 2025. URL https://arxiv.org/abs/2501.12326.

Pascal J. Sager, Benjamin Meyer, Peng Yan, Rebekka von Wartburg-Kottler, Layan Etaiwi, Aref Enayati, Gabriel Nobel, Ahmed Abdulkadir, Benjamin F. Grewe, and Thilo Stadelmann. A Comprehensive Survey of Agents for Computer Use: Foundations, Challenges, and Future Directions. *Journal of Artificial Intelligence Research*, 85, 2026. doi: 10.1613/jair.1.19490. URL https://doi.org/10.1613/jair.1.19490.

Qingyang Shao, Xin Zhang, Zhouyang Yuan, Xianying Chen, Yujia Xiang, Zihao Yang, Tong Ye, Yangqi Zhang, Jiakang Xu, Xiaoqing Yan, et al. Bioinfoysis technical report. *arXiv preprint arXiv:2609.03871*, 2026.

Shuaike Shen, Wenduo Cheng, Mingqian Ma, Alistair Turcan, Martin Jinye Zhang, and Jian Ma. SKILLFOUNDRY: Building Self-Evolving Agent Skill Libraries from Heterogeneous Scientific Resources. *arXiv preprint arXiv:2604.03964*, 2026. URL https://arxiv.org/abs/2604.03964.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

Qiushi Sun, Zhoumianze Liu, Chang Ma, Zichen Ding, Fangzhi Xu, Zhangyue Yin, Haiteng Zhao, Zhenyu Wu, Kanzhi Cheng, Zhaoyang Liu, Jianing Wang, Qintong Li, Xiangru Tang, Tianbao Xie, Xiachong Feng, Xiang Li, Ben Kao, Wenhai Wang, Biqing Qi, Lingpeng Kong, and Zhiyong Wu. ScienceBoard: Evaluating Multimodal Autonomous Agents in Realistic Scientific Workflows. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/7a9745f251508a053425a256490b0665-Abstract-Conference.html.

Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. SEAgent: Self-Evolving Computer Use Agent with Autonomous Learning from Experience. *arXiv preprint arXiv:2508.04700*, 2025. URL https://arxiv.org/abs/2508.04700.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5d413e48f84dc61244b6be550f1cd8f5-Abstract-Datasets_and_Benchmarks_Track.html.

Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. *arXiv preprint arXiv:2605.23904*, 2026. URL https://arxiv.org/abs/2605.23904.

Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue (Steve) Liu, Xiaoxiao Li, and Philip S. Yu. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. *arXiv preprint arXiv:2604.01687*, 2026. URL https://arxiv.org/abs/2604.01687.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM Agents Are Experiential Learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19632–19642, 2024. doi: 10.1609/aaai.v38i17.29936. URL https://ojs.aaai.org/index.php/AAAI/article/view/29936.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=oKn9c6ytLx.

Appendix Overview

This appendix contains the following parts:

- ★ **Appendix A: Discussion and Implications Recommended reading** — applicability, knowledge organization, and base model capabilities.
- **Appendix B: Direct Documentation Injection Details** — model-based injection protocol, context-window constraints, and selected documentation statistics.
- **Appendix C: Retrieval-Augmented Generation Details** — offline corpus preparation, structure-preserving chunking, hybrid retrieval with reranking, and frozen task bundles.
- **Appendix D: SkillFoundry Reproduction Details** — reproduction of SkillFoundry as a Resource-to-Skill baseline, including setup, inputs, and procedure.
- **Appendix E: VisualSkill Reproduction Details** — reproduction of VisualSkill as a Resource-to-Skill baseline, including setup, inputs, and procedure.
- **Appendix F: Resource-to-Skill Implementation Details** — pipeline details, including child-session contracts and deterministic validation.
- **Appendix G: ScienceBoard-Verified: Detailed Repairs** — audited revisions: instruction ambiguity, task and answer errors, evaluator bias, and environment repairs.
- **Appendix H: Task Construction and Deduplication** — construction of the 108 evolution tasks and deduplication against the test set.
- **Appendix I: Execution-Guided Skill Self-Evolution: Additional Details** — history format, failure diagnosis, validation, submission, and consolidation protocols.
- **Appendix J: Case Study: Complementary Roles of Skills and References in KAlgebra** — three-way C-05 comparison of script-loading guidance, expression knowledge, and task completion.
- **Appendix K: Case Study: How Self-Evolve Repairs Resource–Environment Mismatches and Improves Task Performance** — two KAlgebra skill revisions, from execution failure to transfer.
- **Appendix L: Case Study: Adding a Module-Level Reference for GRASS GIS** — addition of task-relevant reference knowledge and its selection on related test tasks.
- **Appendix M: Case Study: Outdated Interface Guidance in Celestia** — how guidance inherited from an outdated source interface consumes execution steps and causes task failure.
- **Appendix N: Task Requirements and Documentation Scope for Lean 4 and TeXstudio** — Verified task requirements compared with the source documentation used for the initial libraries.
- **Appendix O: Research Setting and Baseline Reproduction Constraints of Resource2Skill** — research setting, knowledge objectives, and limitations in adapting the released implementation.
- **Appendix P: Supplementary Evaluation of Frontier Models** — model performance and gains from the self-evolved ScienceCUA library.

A Discussion and Implications

Niche and private software are priority settings for our approach. Knowledge of niche, private, or self-developed software may be underrepresented during general models' pre-training and post-training, leaving gaps in their understanding of specific functions, workflows, and usage constraints. Extracting knowledge from documentation therefore has the potential to yield larger performance gains for such software. Execution feedback can then guide revisions and additions that adapt this knowledge to the deployed environment and task requirements. For widely used software whose operational knowledge is well covered during training, models may already know much of the documented material, leaving less room for improvement through external knowledge.

Knowledge organization should reflect software and task requirements. Scientific software requires both operational workflows and precise parameter constraints, syntax rules, and command and function catalogs. These details are often shared across tasks and consulted only when a step requires them. ScienceCUA therefore organizes goal-oriented procedural knowledge as Skills and shared lookup knowledge as References, allowing agents to follow workflows while accessing specific rules and details on demand. Other software and scenarios may require different forms of knowledge organization. Applying our approach to these settings can retain the process of constructing knowledge from resources and improving it through execution, while adapting knowledge organization to the target setting. Such adaptation could include exploring code-based or visual representations, whose effectiveness we have not systematically compared within the same framework.

Base model capabilities affect the value of external knowledge augmentation. Recent frontier models, such as GPT-6 Astra, demonstrate stronger computer-use capabilities. As models become able to complete more scientific-software tasks on their own, the additional gains from ScienceCUA's self-evolved Skills and References may diminish, a pattern observed in our supplementary experiments (Appendix P). Similar effects may arise in other CUA settings: when a base model already possesses the relevant operational knowledge, the room for improvement from Skills may shrink; if their guidance is outdated, inapplicable to the task, or imposes unnecessary constraints on execution paths, it may even interfere with the model's own judgment and lead to worse performance than the base model without Skills. The value of external knowledge therefore needs to be evaluated in relation to the base model's capabilities and the specific task. Practical deployment must also balance task performance against inference cost. In budget-constrained settings that call for lower-cost models, constructing and continually improving external knowledge from software resources retains practical value. Moreover, the software covered by ScienceBoard has publicly available documentation, and models may already have learned some of this operational knowledge. For niche, private, or self-developed software whose knowledge is underrepresented during model training, external knowledge may still fill capability gaps and yield larger gains. This expectation warrants further empirical validation.

B Direct Documentation Injection Details

Direct injection follows the model-based evaluation protocol in Section 4.3. Each API request includes the system prompt, task instruction, screenshots and accessibility trees from the three most recent steps together with the model-generated actions for those steps, and the current screenshot and accessibility tree. Documentation for the corresponding application is added to every request. ScienceBoard executes the action code or control instructions returned by the model.

Direct injection is constrained by the model's context window. Table 4 lists the officially documented context limits of the four evaluated models. These windows must accommodate documentation, task instructions, interaction history, and other inputs while leaving room for model output; they cannot be allocated entirely to documentation.

Table 4: Official context-window limits of the evaluated models.

Model	Context window	Official source
Qwen-3.8 Max	1M tokens	Qwen documentation
GLM-5.2	1M tokens	Z.AI documentation
Grok-4.6	500,000 tokens	Grok model documentation
GPT-5.5	1,050,000 tokens	OpenAI model documentation

The complete documentation for some applications also contains substantial amounts of text, requiring considerable context space for full injection. As shown in Table 5, the full GRASS GIS and ChimeraX documentation contains approximately 12.2 million and 1.7 million text characters, respectively. Injecting these materials in full at every step, alongside task and interaction inputs, creates a risk of exceeding the context window.

Table 5: Text sizes of the full documentation and the documentation selected for direct injection.

Software	Format	Full documentation (characters)	Selected documentation (characters)
KAlgebra	PDF	27,572	27,572
Celestia	PDF	146,761	146,761
GRASS GIS	HTML	12,232,449	367,177
ChimeraX	HTML	1,704,478	254,697
Lean 4	HTML	527,662	527,662
TeXstudio	HTML	243,645	243,645

Full documentation refers to all materials within the documentation scope specified in this work. Both columns use the same character-counting procedure: text is extracted from PDF text layers or HTML with tags removed, whitespace is normalized, and characters are counted including whitespace. Navigation text is retained for HTML.

Given these context limits and documentation sizes, we use a fixed documentation configuration for each application. KAlgebra and Celestia receive their complete original PDF files. For Lean 4 and TeXstudio, we acquire all pages within the specified documentation scope and inject their normalized text in full. For GRASS GIS and ChimeraX, we use bounded page acquisition: the root page is retained, and 40 child pages are selected in traversal-depth order with round-robin scheduling across root-level chapter branches. The normalized text of all selected pages is then injected in full. All documentation is fixed before evaluation, shared across tasks and evaluated models for the same application, and included in full in every execution request, without task-specific retrieval or dynamic selection during execution.

C Retrieval-Augmented Generation Details

This appendix describes the retrieval-augmented generation (RAG) pipeline. Offline preprocessing freezes one corpus per application, applies structure-preserving chunking and hybrid retrieval, and materializes task-specific context bundles for deterministic loading at execution. All tasks of an application share the same corpus. Documents are never manually paired with tasks.

C.1 Frozen Documentation Corpora

We construct one physically isolated corpus for each of the six applications. Table 6 reports its documentation input and size. Raw inputs, normalized text, source locators, acquisition records, and manifests are all frozen before retrieval. Bundle generation and benchmark execution use only these local snapshots, shared across all tasks and evaluated models for an application.

Table 6: Frozen documentation corpora used by RAG. Documents are semantic sections when reliable boundaries are available and page-level fallbacks otherwise. GRASS GIS and ChimeraX counts cover all successfully acquired pages.

Software	Environment	Format	Frozen documentation source	Pages	Documents	Chunks
KAlgebra	24.11.70	PDF	*KAlgebra Handbook*	32	88	96
Celestia	1.7.0	PDF	*Celestia 1.6.1 User's Guide*	48	48	57
GRASS GIS	8.4.1	HTML	GRASS GIS 8.4.1 Reference Manual	1,468	11,444	24,236
ChimeraX	1.9	HTML	UCSF ChimeraX online User Guide, frozen at acquisition	277	726	859
Lean 4	4.14.0	HTML	*Theorem Proving in Lean 4*, version 4.26.0	13	113	146
TeXstudio	4.8.6	HTML	TeXstudio 4.9.8 documentation	9	269	294
Total				**1,847**	**12,688**	**25,688**

PDF normalization. PDF text is extracted with PyMuPDF from the embedded text layer. OCR is disabled and raster images are not indexed. Repeated headers and footers are removed. We infer a hierarchy from chapter, section, appendix, and numbered headings and retain page ranges as source locators. When a reliable semantic heading is unavailable, the corresponding page is kept as a page-level document rather than discarded. This yields 81 semantic sections and 7 page fallbacks for KAlgebra. The Celestia guide is retained as 48 page-level documents because its layout does not provide sufficiently reliable semantic section boundaries.

HTML acquisition and freezing. Each HTML corpus begins at a fixed official documentation root. URLs are canonicalized and restricted to the documentation scope. The reachable in-scope link queue is traversed to exhaustion, with no page-count limit, and every successful response is frozen with its requested URL, final URL, acquisition time, raw HTML hash, outgoing links, and local file locator. GRASS GIS contains 1,468 successful pages and ChimeraX contains 277. Both acquisition queues have zero pending URLs. Images, video, CSS, JavaScript, PDFs, and pages outside the documentation scope are excluded. The Lean 4 and TeXstudio documentation trees contain 13 and 9 in-scope pages, respectively, all of which are retained.

HTML normalization. For every retained page, the extractor selects the `main`, `article`, or body content and removes scripts, styles, templates, and navigation chrome. It segments the remaining content at `h1`–`h6` boundaries. Paragraphs, lists, tables, and preformatted code are preserved in reading order, with code represented as fenced blocks. Exact duplicate pages are removed by content hash. Raw HTML and normalized Markdown-compatible text are both retained for audit, but only normalized text is indexed. This procedure is deterministic and uses no language model.

C.2 Structure-Preserving Chunking

Chunk boundaries are computed with the tokenizer of `BAAI/bge-m3`. Each indexable text is prefixed with its document title and complete section breadcrumb. The target length is 700 tokens and the hard maximum, including this prefix, is 1,000 tokens. Text that exceeds the limit is split preferentially at paragraph, line, or sentence boundaries with an 80-token overlap between consecutive pieces. Chunks never cross documents or semantic sections. Exact duplicates are removed using the SHA-256 hash of the complete index text. Near-duplicate removal is not applied. The counts in Table 6 are post-deduplication, and no resulting chunk exceeds the hard limit.

C.3 Hybrid Retrieval and Reranking

Retrieval is performed independently for each application. The query is exactly the stripped task instruction. The software name is used only to select the corresponding corpus and is not appended to the query. Cross-application retrieval is disabled.

We retrieve 30 candidates with BM25 and 30 with a dense encoder. BM25 uses $k_1 = 1.2$, $b = 0.75$, binary query term frequency, and a Unicode-aware tokenizer that preserves compound identifiers. Dense retrieval uses `BAAI/bge-m3` with a maximum input length of 1,024 tokens, CLS pooling, and L2-normalized embeddings. Encoding uses FP16 on CUDA with batch size 2. The resulting 1,024-dimensional vectors are stored in FP32. Similarity is exact cosine similarity, implemented as a dot product over normalized vectors, without approximate-nearest-neighbor search.

The two ranked lists are combined by equal-weight reciprocal rank fusion (RRF):

$$\mathrm{RRF}(d) = \sum_{s \in \{\mathrm{BM25,dense}\}} \frac{1}{60 + \mathrm{rank}_s(d)}. \tag{1}$$

The 40 highest-scoring fused candidates are reranked with `BAAI/bge-reranker-v2-m3`. The reranker uses a maximum input length of 1,024 tokens, FP16 CUDA inference, batch size 4, and its unnormalized output score. Ties are resolved deterministically using the fused score and chunk identifier.

We select up to five reranked chunks whose rendered references fit a 6,000-token context budget. Each numbered reference gives its rank, document title, section breadcrumb, and chunk text. Retrieval scores are hidden. The references are enclosed by documentation markers and prepended to the unmodified original task instruction.

C.4 Frozen Task Bundles and Runtime Protocol

Offline retrieval produces one frozen bundle per ScienceBoard-Verified task, storing the query, corpus and index identities, configuration hash, candidate rankings, final five chunks, and rendered instruction. The 168 bundles comprise 31 for KAlgebra, 33 for Celestia, 34 for GRASS GIS, 28 for ChimeraX, 26 for Lean 4, and 16 for TeXstudio. Each is reused across all evaluated models.

At initialization, hashes and identities are verified. Invalid bundles fail without live fallback. Runtime uses no retrieval assets or evaluation data.

D SkillFoundry Reproduction Details

This appendix describes our reproduction of SkillFoundry as a Resource-to-Skill baseline.

D.1 Reproduction Setup

We reproduced the complete SkillFoundry pipeline from an empty repository state, without an external taxonomy or any pre-existing resource or skill library. Each target software package was processed in an isolated repository, and no task list or task-specific resource pairing was provided. Under this setting, SkillFoundry was responsible for constructing the software-specific taxonomy, discovering supporting resources, and building reusable skills from the retrieved evidence.

Table 7: Inputs used for the SkillFoundry reproduction.

Software	Environment version
KAlgebra	24.11.70
Celestia	1.7.0
GRASS GIS	8.4.1
Lean 4	4.14.0
TeXstudio	4.8.6
ChimeraX	1.9

D.2 Inputs and Configuration

Each run received only the software name and environment version. Table 7 lists the six targets used in our experiments. We used the same execution configuration for all targets: stage-workers=1, background-validation-limit=0, and verification-mode=standard.

D.3 Reproduction Procedure

For each target, we executed one complete five-stage cycle in the standard order: `tree_check`, `resource_search`, `skill_build`, `skill_test`, and `refresh`. The pipeline first constructed a software-specific capability taxonomy, then searched for and registered relevant external resources. It subsequently synthesized skills from the registered evidence, ran SkillFoundry's repository-local validation, and synchronized the resulting taxonomy, resource registry, and skill registry. The generated skills were then evaluated on ScienceBoard-Verified using the same downstream protocol as the other Resource-to-Skill methods.

E VisualSkill Reproduction Details

This appendix describes our reproduction of VisualSkill as a Resource-to-Skill baseline.

E.1 Reproduction Setup

We reproduced VisualSkill's standard multimodal Stage 1 pipeline rather than its task-filtered mode. Each run processed one target application with exactly one curated documentation source, either a PDF manual or an HTML documentation root. PDF and HTML inputs were not mixed within a run, no external task list was provided for taxonomy filtering, and the optional text-derivation phase was not used.

E.2 Inputs and Configuration

Table 8 lists the documentation source supplied for each target, with public links to the documentation resources.

Table 8: Inputs used for the VisualSkill Stage 1 reproduction.

Software	Environment version	Resource version	Format	Documentation source
KAlgebra	24.11.70	20.12	PDF	https://docs.kde.org/stable_kf6/en/kalgebra/kalgebra/kalgebra.pdf
Celestia	1.7.0	1.6.1	PDF	https://drive.google.com/file/u/1/d/1dxMnahknnVjrckqWOyJhvuS8IA0BCT0l/view
ChimeraX	1.9	1.9.0	HTML	https://web.archive.org/web/20241215145343id_/https://www.cgl.ucsf.edu/chimerax/docs/user/index.html; root prefix /chimerax/docs/user/
GRASS GIS	8.4.1	8.4.1	HTML	https://lira.no-ip.org:8443/doc/grass-doc/html/index.html
Lean 4	4.14.0	4.26.0	HTML	https://lean-lang.org/theorem_proving_in_lean4/; root prefix /theorem_proving_in_lean4/
TeXstudio	4.8.6	4.9.8	HTML	https://texstudio-org.github.io/

We used `--mode multimodal` for all six targets. PDF sources were supplied through `sources.pdf_guide.url`, whereas HTML sources were supplied through `sources.html_guide.root_url`; otherwise, the same Stage 1 construction setting was used across targets.

E.3 Reproduction Procedure

For each target, we ran the standard Stage 1 construction pipeline. VisualSkill first built a category–topic taxonomy from the supplied documentation, then prepared topic-level textual and visual evidence from the corresponding PDF pages or HTML pages. It next generated a multimodal `guide.md` for each topic and produced a `use_when` routing description. These topic-level artifacts were finally assembled into the top-level `SKILL.md`. The generated skills were then evaluated on ScienceBoard-Verified using the same downstream protocol as the other Resource-to-Skill methods.

F Resource-to-Skill Implementation Details

This appendix provides implementation details for the Resource-to-Skill pipeline in Sec. 3.1. The main text focuses on the knowledge-construction logic; here we report the executable child-session contracts and deterministic validation. Lower-level parser corner cases, full tool JSON schemas, retry/error handling, and storage internals are left to the released implementation.

F.1 DSH Child Session Contracts

A DSH child session is defined by more than its prompt. For an Agent a, the complete executable contract is

$$\mathfrak{C}_a = (P_a, I_a, T_a, B_a, O_a, V_a, \tau_a), \tag{2}$$

where P_a is the prompt, I_a is the input scope, T_a is the allowed tool surface, B_a is the tool budget, O_a is the structured output contract, V_a is the Python validator, and τ_a is the termination protocol. A child session can observe only authorized objects exposed through T_a and can modify pipeline state only through the designated structured submission tools.

The HTML and PDF Evidence Architect contracts are source-type-specific and mutually exclusive. The HTML child accepts only one normalized HTML root scope, comprising the root page and all eligible in-scope child pages; the PDF child accepts only one normalized PDF document. Neither child may be dispatched a source of the other type.

Table 9: Executable contracts for the Resource-to-Skill child sessions. Budgets refer to the default production configuration; protocol-reserved calls are listed separately where applicable.

Stage	Executable contract
Evidence Architect (HTML)	**Input.** One normalized HTML root scope (root page plus all eligible in-scope child pages); root-authored structure; candidate Evidence skeleton **Tools.** inspect_source, get_root_structure, submit_evidence_revision **Output / budget.** EvidenceRevision; exactly 3 calls in the default workflow
Evidence Architect (PDF)	**Input.** One normalized PDF document; selected navigation tree; content assignments; residual content; candidate skeleton **Tools.** inspect_source, get_pdf_document_structure, bounded read_pdf_content and read_evidence_node, submit_evidence_revision **Output / budget.** EvidenceRevision; at most 6 calls
Reference Build	**Input.** Frozen CorpusSnapshot and Python-generated review manifest **Tools.** Candidate/catalog inspection, bounded Evidence reading, submit_reference_review **Output / budget.** Reference review decisions; at most 8 calls
Task Discovery	**Input.** Frozen CorpusSnapshot **Tools.** Evidence inventory, search/read/image inspection, Task batch submission **Output / budget.** TaskDraft; exploration budget $\max(24, \min(40, 3\,\texttt{max_steps}))$
Trajectory Inference	**Input.** One validated Task plus Corpus/Knowledge snapshots **Tools.** Evidence, Task, Reference and Reference-entry reads; trajectory submission **Output / budget.** TrajectoryDraft; $\max(24, 2\,\texttt{max_steps})$ plus 2 protocol calls
Skill Induction	**Input.** A batch of validated Trajectories plus KnowledgeSnapshot **Tools.** Upstream-object reads, Skill submission, coverage inspection/review **Output / budget.** SkillDraft; $\max(40, 3\,\texttt{max_steps})$ plus 6 protocol calls

F.2 Deterministic Validation and Control

All Agent outputs are validated by Python before entering a downstream stage. A textual citation must contain a quote that exactly matches the corresponding Evidence text; a visual claim must reference an authorized and inspected image artifact. Citations cannot point to nonexistent or out-of-scope Evidence, and downstream objects can reference only upstream identifiers present in the current frozen snapshot.

Stage-specific validation additionally checks schema constraints, duplicate candidates, field lengths, step ordering, and cross-object consistency. For Trajectories, Python verifies that the submitted task_id matches the dispatched Task, that exactly one acceptable trajectory is returned,

that step ordinals are consecutive, and that linked References exist and satisfy the reference_guided dependency rules. For Skills, Python validates Task/Trajectory/Reference relations, step structure, exact citations, and the coverage protocol over every dispatched Trajectory.

Pipeline state changes only through structured submission tools. Ordinary natural-language replies do not commit business results. Key stages freeze immutable CorpusSnapshot and KnowledgeSnapshot objects so that downstream child sessions read stable inputs. Attempt/Receipt bookkeeping, idempotent retry, recovery, and storage details are implementation mechanisms and are documented in the released code rather than expanded here.

G ScienceBoard-Verified: Detailed Repairs

This appendix documents the 37 task revisions and one task removal made when constructing ScienceBoard-Verified, corresponding to the 38 revised or removed tasks summarized in Figure 5. A task may involve multiple issues; we assign it to one category to avoid double counting and describe secondary changes in the same entry. The first three categories contain 15, 12, and 10 revised tasks, respectively. The fourth contains the removed task and shared evaluation-environment repairs, which are not counted as additional task revisions.

G.1 Instruction Ambiguity

These revisions clarify the intended target, information source, or completion requirements so that the instructions explicitly state what the evaluator expects.

1. Celestia A-02: Time direction and rate. The original instruction, "-20x slower," could be interpreted as running backward at 20 times normal speed, whereas the evaluator expects a time scale of 0.05. The revised instruction explicitly requests forward playback at 0.05x, or 20 times slower than real time.

2. Celestia C-06: Ambiguous G0 star selection. The instruction asks for the name of the "only" G0 star, but the software lists multiple stars of this spectral type. We specify the closest G0 star to identify the intended object unambiguously.

3. Celestia C-10: Unspecified distance convention. Asking for the globular cluster "farthest from the Milky Way" could prompt agents to change the distance reference. The revision instead specifies the cluster with the largest `Distance (ly)` value in Celestia's Deep Sky Objects browser and requests its radius in light-years to two decimal places.

4. TeXstudio A-01: Angle insertion. The task asks the agent to measure an angle between specified atoms in ChimeraX and replace a placeholder in the article. The evaluator checks the saved file and compilation result, but these requirements were omitted from the instruction. We explicitly require saving the TeX file and ensuring successful compilation.

5. TeXstudio A-02: Residue-name insertion. The task asks for the name of residue `/A:6` to be inserted into a table. We add the previously unstated requirements to save the TeX file and ensure that it compiles successfully.

6. TeXstudio A-03: DNA sequence correction. The task asks the agent to correct the Chain A DNA sequence using ChimeraX. We explicitly require saving the corrected TeX file and successful compilation, matching the conditions checked by the evaluator.

7. TeXstudio A-04: Fibonacci value correction. The task asks the agent to obtain the Fibonacci value $f(24)$ in KAlgebra and replace an incorrect value in the article. The revision adds saving and successful compilation as explicit completion requirements.

8. TeXstudio B-01: Lean 4 citation. The task asks the agent to fill a citation in `main.tex` using the existing Lean 4 entry in `refs.bib`, without modifying the bibliography file. We preserve these constraints and add the requirements to save the TeX file and ensure successful compilation.

9. TeXstudio B-02: COSPAR identifier and citation. The task asks for Gemini 9A's COSPAR identifier followed immediately by a Celestia citation. We clarify that the completed TeX file must be saved and compile successfully.

10. TeXstudio B-03: Province count and citation. The task asks the agent to insert the number of provinces in Piemonte and a GRASS GIS citation into the article. We add the missing requirements to save the TeX file and ensure successful compilation.

11. TeXstudio C-01: Atom-count correction. The task asks the agent to correct a C, O, or N atom count using ChimeraX. We explicitly require saving the corrected TeX file and successful compilation.

12. TeXstudio C-02: Single-letter correction. The task asks the agent to correct a typo in the deduction rules of the λC type system by changing exactly one letter. We retain this restriction and add saving and successful compilation to the instruction.

13. TeXstudio C-03: Figure-path repair. The task asks the agent to repair the incomplete figure path `figs/lat`. We add the requirements to save the TeX file and verify successful compilation.

14. TeXstudio C-04: Figure width and completion requirements. The original English instruction misspells "width" as "with." We correct the typo, retain the restriction to change only the width of `figs/envelope` to 0.48, and explicitly require saving and successful compilation.

15. TeXstudio C-05: Lean code correction. The task asks the agent to correct Lean 4 code in the article by changing exactly one word, with the supplied VS Code available for debugging. We correct "exact one word" to "exactly one word" and add the requirements to save the TeX file and ensure successful compilation.

G.2 Task & Answer Errors

These repairs correct task statements, reference answers, labeled test points, and expected results encoded in evaluators, including mathematical statements with insufficient assumptions.

16. KAlgebra C-07: Evaluating the wrong derivative. The task asks for the derivative of $f(x) = \ln(x+1) + x^2 - x$, but the original evaluator uses labels corresponding to the derivative of $\ln(x^2 + 1) + x^2 - x$. We rebuild the test points using

$$f'(x) = \frac{1}{x+1} + 2x - 1,$$

with positive examples inside the original function's domain.

17. KAlgebra C-12: Incorrect positive points for a distance-product curve. The product of the Euclidean distances to $(1, 0)$ and $(-1, 0)$ must equal 16, giving the implicit equation

$$\big((x-1)^2 + y^2\big)\big((x+1)^2 + y^2\big) = 256.$$

The original positive examples do not lie on this curve. We recompute their coordinates according to the stated condition.

18. KAlgebra C-14: Confusing diameter with radius. The stated diameter is $2\sqrt{2}$, so the radius is $\sqrt{2}$ and the sphere satisfies $x^2+y^2+z^2 = 2$. The original labels instead correspond to $x^2+y^2+z^2 = 8$, treating the diameter as the radius. We replace the test points to distinguish the requested sphere from the incorrect one.

19. Celestia C-09: Apparent-magnitude answer inconsistent with the software. The original reference answer is 4.47. In the benchmark's initial environment, sorting the galaxy list by distance and excluding the Milky Way selects Sagittarius dSph, whose displayed apparent magnitude is 3.84. We update the answer to 3.84 and specify that the answer must come from Celestia's Celestial Browser, excluding the Milky Way itself.

20. Celestia D-02: Eclipse date and answer format. The original answer gives December 21 for the last lunar eclipse of 2056; we correct it to December 22. The revised instruction also specifies the UTC date at maximum eclipse as seen from Earth and requests only the numeric day of December, removing ambiguity about the event time and submission format.

21. ChimeraX C-04: Incorrect hydrogen count. The task asks for the number of hydrogen atoms in `/A:1`. We correct the reference answer from 12 to 0 and update the evaluator accordingly.

22. Lean 4 A-02: Overly strong Ackermann inequality. The original statement requires $\mathrm{ack}(x, y+1) < \mathrm{ack}(x+1, y)$. Equality can occur when $y = 0$, so the strict inequality is false. We replace $<$ with $\leq$.

23. Lean 4 C-05: Incorrect set-containment assumptions. The original assumptions $A \subseteq Y$ and $Y \subseteq A^{\mathsf{c}}$ do not imply that Y is compact. For example, when $A = \emptyset$, they permit noncompact closed sets Y. We replace these containment conditions with $Y \subseteq A$, retaining compactness of A and closedness of Y, so that the statement expresses compactness of a closed subset of a compact set.

24. Lean 4 D-02: Missing convexity assumption. The original statement tries to derive uniform continuity on S from a bounded derivative on S, without conditions connecting its points. We add `Convex` $\mathbb{R}$ S, allowing the derivative bound to control function differences through a mean-value inequality and support the uniform-continuity conclusion.

25. Lean 4 D-07: Moment condition does not express integrability. The original statement binds X twice and uses an equality between an expectation and some real number M without explicitly requiring the necessary integrability. We remove the duplicate binding and assume that $\omega \mapsto |X(\omega)|^p$ is integrable with respect to $\mathbb{P}$, supporting the tail conclusion

$$x^p\,\mathbb{P}\big(|X| > x\big) \longrightarrow 0 \qquad (x \to +\infty).$$

26. Lean 4 E-03: Hamiltonian endpoints and edge bound. The original hypothesis requires a Hamiltonian walk between every pair of vertices, including identical endpoints. We restrict it to distinct vertices. The extra constant term $+1$ in the original conclusion is also invalid: K_4 satisfies the revised hypothesis but has $2e = 3n$. We therefore correct the conclusion to $2e \geq 3n$ for $n \geq 4$.

27. GRASS GIS D-01: Incorrect expected schema type. The task requests a `COLOR varchar` column, which the current GRASS GIS and SQLite environment reports as `TEXT` through `v.db.select`. The original evaluator instead expects `CHARACTER`. We retain the requested `varchar` declaration and change the expected schema type to `TEXT`. The evaluator already supports both GUI and Console execution; this repair corrects the expected result.

G.3 Evaluator Bias

These repairs remove unnecessary evaluator restrictions on execution paths, output representations, and output locations while preserving the task requirements. For GRASS GIS B-03–B-10, we verified that both GUI operations and Console commands achieve the required outcomes. However, the original evaluators rely on GUI property caches and reject equivalent Console executions when those caches are not populated. We extend the checks to recognize both paths. TeXstudio B-04 addresses overly strict matching of BibTeX title capitalization and formatting. GRASS GIS B-12 extends output checking to a separate statistics window. Tasks that explicitly require a GUI state change retain checks of that state.

28. GRASS GIS B-03: Category and topology display. The original evaluator reads `display` from the layer-properties cache. An equivalent `d.vect` command can produce the requested display without populating that cache. We inspect either GUI parameters or layer command state for both `cat` and `topo`, excluding states marked hidden or unchecked.

29. GRASS GIS B-04: Area fill color. The task requires the fill color $(191, 191, 191)$. In addition to GUI parameters, we inspect `fill_color` or `fcolor` in the layer's `d.vect` command. The evaluator accepts RGB notation and the equivalent RGBA notation with an opaque alpha value.

30. GRASS GIS B-05: Road line color. The task requires white lines for `roadsmajor`. We inspect GUI parameters or layer command state and accept `white`, `255:255:255`, and the equivalent opaque RGBA representation.

31. GRASS GIS B-06: Railway line width. A Console command can set the `railroads` line width to 2 while leaving the GUI parameter cache empty. We add checks of layer command state and accept both `2` and `2.0`.

32. GRASS GIS B-07: Fire-station symbol. The task requires the `basic/marker` symbol for `firestations`. We check the `icon` parameter in either GUI properties or the layer's `d.vect` command, allowing an equivalent Console result to be recognized.

33. GRASS GIS B-08: Hospital symbol size. The task requires symbol size 10 for `hospitals`. We additionally inspect the command's `size` parameter and accept both `10` and `10.0`, removing dependence on editing the GUI property panel.

34. GRASS GIS B-09: School filtering condition. The task requires displaying schools satisfying the wheelchair-access condition. We inspect `where` in either GUI parameters or layer command state and remove whitespace before comparison with `wheelchair='yes'`.

35. GRASS GIS B-10: Label configuration. The task requires the `boundary_region` label field `NOME`, font `Ubuntu:Bold`, and horizontal alignment `center`. Console execution can produce the requested display while leaving the GUI cache empty. We check all three settings in either GUI parameters or the same layer command, recognizing both completion paths.

36. GRASS GIS B-12: Statistics in a separate output window. The original evaluator searches only `dump['output']` for `n: 263975`. We retain that check and add a fallback for a separate `r.univar` window. The fallback uses the accessibility tree to identify the target map and selected Command output tab, then checks the corresponding image region for rendered output, covering cases where the output is absent from the original field.

37. TeXstudio B-04: Overly strict BibTeX title matching. The task asks the agent to obtain a paper's BibTeX from Google Scholar and write it to `refs.bib`. The original evaluator requires exact title capitalization and spacing around the equals sign, rejecting valid entries for the same paper with equivalent textual representations. We make title matching case-insensitive and allow whitespace around the equals sign. The instruction also explicitly requires saving `refs.bib`.

G.4 Environment Repairs

This category covers repairs to the evaluation environment, including application-usage prompts, answer-submission protocols, action-response handling, compilation, and initialization settings. It also records the removal of one task requiring an unavailable external account. These shared repairs are not counted as additional task revisions.

38. ChimeraX D-03: External account required for AlphaFold. The task asks for an AlphaFold structure prediction from a supplied amino-acid sequence. Completion requires an external Google account that is not provided by the benchmark environment. We remove this task from the final evaluation set.

Answer-submission protocol (32 tasks). We observed correct answers being rejected because their response format was not correctly handled by the framework. The original prompt did not sufficiently specify the code-block structure required for `ANS` submissions. This protocol issue affects all 32 answer-submission tasks. We explicitly require `ANS` followed by the actual answer inside a separate fenced code block, with neither the answer on the fence line nor additional text outside the block.

Application-usage prompts. The original `LEAN_VM_USAGE` incorrectly contains TeXstudio document-editing instructions, while `TEXSTUDIO_VM_USAGE` is undefined. We provide the correct Lean 4 instructions: complete the target proof or code in VS Code, save the file, and use Lean feedback to check for errors, warnings caused by `sorry`, and remaining goals. We also restore the TeXstudio-specific document-editing and compilation instructions.

Malformed action responses. The original framework does not validate unsupported tool or scratchpad control markers, so responses containing duplicated actions inside such envelopes can be parsed as ordinary action text. We reject these responses before emitting any actions and allow one retry with the same observation. The recovery prompt asks the agent to follow the existing action format without unsupported envelopes or duplicated draft and final content.

TeXstudio compilation environment. We repair the TeXstudio compilation environment so that compilation checks can run normally, preventing an environment failure from blocking task completion and evaluation.

GRASS GIS initial pointer position. At initialization, the mouse pointer originally obscured the Console area, interfering with observation and evaluation. We reposition it near the center of the interface so that the Console remains unobstructed.

Update notifications. An update dialog appeared after every initialization. Dismissing it consumed interaction steps and materially interfered with evaluation by using part of the task's step budget on an unrelated action. We disable updates to prevent these recurring dialogs and their associated evaluation overhead.

H Task Construction and Deduplication

We construct 108 additional tasks across the six scientific applications for skill self-evolution. These tasks use the same types of software operations as the test tasks to support the transfer of operational experience, while varying the target objects, task parameters, or specific requirements to form distinct tasks. We manually compare each additional task against the ScienceBoard-Verified tasks, examining its goal, initial state, required output, and evaluation criteria to confirm that it does not duplicate a test task. Researchers in the corresponding scientific domains who are familiar with the respective software manually review and execute all additional tasks to verify that their requirements are clear, their reference answers or target states are correct, and they can be completed in the specified software environments.

I Execution-Guided Skill Self-Evolution: Additional Details

This appendix specifies the execution protocol used in Section 3.2. Evolution uses only $E = \text{Expanded} \setminus \text{Verified}$ and the original resources $\mathcal{D}$. For new-knowledge creation, $\mathcal{D}$ is accessed through bounded live browsing that starts from the configured official documentation root for the current application and follows only in-scope links discovered from fetched official pages. It does not use a search engine, BM25 retrieval, a saved Resource-to-Skill snapshot, or model knowledge as source evidence. The execution model is fixed, and $V = \text{Verified}$ is reserved for final evaluation.

I.1 Frozen Initial Library and Per-Task State

All evolution tasks start from the same frozen initial library $\mathcal{L}_0 = (\mathcal{S}_0, \mathcal{R}_0)$. A task never installs a provisional update into the library used by another task. Instead, it maintains a private execution history and may contribute at most one independently validated candidate to a global candidate pool $\mathcal{P}$. Candidates are consolidated only after all tasks terminate.

For each task q, its first task exploration selects a fixed routed set $\mathcal{F}_q$ of Skills and References from $\mathcal{L}_0$. Later task explorations use the same routed names and initial versions, even after the task has produced a failed candidate. This separates behavioral exploration from candidate installation: rejected knowledge remains evidence, not active guidance.

Each candidate contains one complete target entry relative to $\mathcal{L}_0$. A candidate produced after a failed exploration may revise one entry in $\mathcal{F}_q$ or add one Skill or Reference supported by $\mathcal{D}$. A candidate distilled after a later successful exploration may only revise one existing routed entry. A rejected candidate is never used as the textual base of a later candidate.

I.2 Complete Execution History and Input Separation

Each task has an indexed history archive. For every task exploration and candidate validation, the archive records the task instruction, execution type, exact execution-time prompt, raw action–observation trajectory, evaluator outcome, routed Skill and Reference names, their complete execution-time contents and version hashes, and every screenshot mapped to its trajectory step. When applicable, it also records the diagnosis and the complete candidate entry used in validation. Screenshots are interpreted as pre-action observations: the effect of an action is checked in the following screenshot.

The first task exploration receives no history. Every later task exploration receives all earlier task explorations and candidate validations for that task, including their exact prompts, raw trajectories, diagnoses, candidates, outcomes, and screenshots. The records are explicitly labeled so that the

agent can distinguish exploration from validation, identify which knowledge version was loaded, and map each screenshot to its execution and step. All historical screenshots are placed in the first model turn and marked as evidence; the final live screenshot remains the state on which the agent must act. The prompt instructs the agent to inspect the complete history, avoid repeating failed action sequences, and not follow initial Skills or References mechanically when current GUI evidence suggests a version mismatch.

Diagnosis and knowledge writing receive the task, loaded entries, exact prompt, raw trajectory, evaluator result, and complete screenshot sequence. The analysis agent must inspect every screenshot before submitting its result. Candidate validation is deliberately separated: it receives no exploration history, successful trajectory, diagnosis, or candidate rationale. This prevents the validation agent from succeeding through transient trajectory memory rather than through the proposed external knowledge.

I.3 Task Exploration, Failure Diagnosis, and Candidate Validation

Let $i \in \{1, \ldots, M_{\text{explore}}\}$ index task explorations for task q. Each exploration begins from the standard initial environment in a fresh session:

$$\tau_q^{(i)} = \text{Execute}_\theta(q; \mathcal{L}_0, \mathcal{F}_q, \mathcal{H}_q^{(i)}), \tag{3}$$

where $\mathcal{H}_q^{(1)} = \emptyset$ and $\mathcal{H}_q^{(i)}$ for $i > 1$ contains all earlier exploration and candidate-validation records for q. Routing is performed normally only for $i = 1$; its result defines $\mathcal{F}_q$ for subsequent exploration and validation.

If $\tau_q^{(1)}$ succeeds, the task exits without proposing an update. A successful exploration with $i > 1$ enters the submission process in Appendix I.4. Each failed exploration is analyzed for one primary root cause. Exhausting the action budget, stopping early, or failing the evaluator describes the observed outcome rather than the underlying cause. The diagnosis uses exactly three categories:

1. **Grounding failure** (`grounding_failure`). Incorrect interface localization, targeting, state reading, or manipulation is the principal obstacle. Repeated grounding mistakes that consume the action budget can qualify. The counterfactual criterion is whether correcting those grounding mistakes would remove the principal obstacle; if an independent knowledge deficiency would remain, a knowledge category must be selected instead.

2. **Modify existing knowledge** (`modify_existing`). One loaded Skill or Reference is relevant but contains an erroneous or incomplete procedure, omitted condition, inappropriate parameter, unsupported syntax, or version-dependent instruction. The target must be one entry in $\mathcal{F}_q$.

3. **Add new knowledge** (`add_new_knowledge`). The routed entries do not contain the capability or operational knowledge needed for the task. This category is reserved for a genuine coverage gap rather than a minor defect in a relevant entry. The new Skill or Reference must be supported by task-relevant content in the original resources $\mathcal{D}$; without sufficient source support, no candidate is produced.

Diagnosis and conditional candidate writing occur in the same analysis session. The diagnosis step fixes the category, evidence, and proposed repair. For `modify_existing`, it also fixes the type and name of one routed target, after which the next step writes its complete replacement while retaining access to the same trajectory and screenshots. For `add_new_knowledge`, the diagnosis deliberately leaves the target type and name unspecified and describes only the missing reusable

knowledge scope. Only after this diagnosis is accepted does the same session unlock bounded live access to $\mathcal{D}$; the research step determines whether the supported artifact is a Skill or Reference, chooses its stable title, and writes one complete new entry. If the fetched official material is insufficient, no candidate is produced. A grounding diagnosis likewise produces no candidate and therefore requires no candidate validation. Either no-candidate outcome consumes the current exploration attempt. If another exploration remains, the task continues with its accumulated history. Even after the final allowed exploration fails, its diagnosis and, when applicable, its one candidate validation are still completed.

Independent Candidate Validation

For a candidate $\Delta_q^{(i)}$, validation constructs a temporary library

$$\widetilde{\mathcal{L}}_q^{(i)} = \mathcal{L}_0 \oplus \Delta_q^{(i)}, \tag{4}$$

where $\oplus$ replaces the target's initial version for a revision or inserts one source-supported entry for an addition. The validation agent starts from the task's standard initial environment in a fresh session. It force-loads the original entries named by $\mathcal{F}_q$; for a revision, the candidate replaces its corresponding original entry, and for an addition, the candidate is force-loaded in addition to all original routed entries. Thus every attempt contains at most one modified or new target.

The validation trajectory is

$$\widetilde{\tau}_q^{(i)} = \mathrm{Execute}_\theta(q; \widetilde{\mathcal{L}}_q^{(i)}, \widetilde{\mathcal{F}}_q^{(i)}, \emptyset), \tag{5}$$

where the empty history excludes all prior trajectories, screenshots, diagnoses, and rationales. The task evaluator alone determines success. A successful candidate is added to $\mathcal{P}$ and the task terminates. A failed candidate, its complete contents, and its validation record are archived; if another task exploration remains, that exploration again uses $\mathcal{L}_0$ and $\mathcal{F}_q$, with the failed validation supplied only as history.

I.4 Submission After History-Guided Success

A task exploration that succeeds at $i > 1$ may have relied on failure history unavailable at deployment. The submission agent therefore contrasts the successful exploration with all earlier failed explorations, candidate validations, loaded knowledge versions, and candidate contents. In one response it either concludes that no knowledge change is warranted or writes a complete revision to one existing entry in $\mathcal{F}_q$. This process cannot create a new Skill or Reference.

At most M_{submit} submission attempts are allowed. Every revised candidate is generated from the target's version in $\mathcal{L}_0$, not by editing the preceding rejected submission candidate. On later attempts, the submission agent additionally receives all previous submission candidates and their validation trajectories so that it can avoid repeating an ineffective revision. Each candidate is validated through the history-free protocol in Equation 5. The first successful candidate enters $\mathcal{P}$ and ends the task.

When the submission agent attributes a failed validation to grounding, it may retain the same candidate and validate it again; the unchanged retry consumes one submission attempt. If no candidate has yet been proposed, it may instead validate the unchanged initial routed knowledge. Success of this unchanged validation ends the task without adding a candidate. If all M_{submit} attempts fail, the task terminates without an update and does not return to task exploration.

I.5 Candidate Consolidation and Final Evaluation

After every evolution task terminates, accepted candidates are grouped by target Skill or Reference. Revisions are grouped by the identity of their original entry. For additions, a conservative Grok semantic-deduplication pass groups candidates only when they have the same knowledge type and express the same reusable capability at substantially the same granularity, scope, and intended use; Skills and References are never grouped together, and uncertain candidates remain separate. Each group adopts the title of one independently validated member rather than inventing a new title. If this auxiliary grouping call is technically unavailable or violates its structured-output contract, the pipeline fails open to exact type-and-title grouping, thereby avoiding an unsupported merge. Different targets are consolidated independently and are never mixed merely because they came from the same task.

If a target has exactly one accepted candidate, that already validated candidate is adopted directly without another task execution. If a target has multiple candidates, the consolidation model receives every accepted candidate and either the target's complete initial content for a revision or an explicit new-entry baseline for an addition. It then produces one complete entry that preserves the independently validated, non-conflicting improvements of all candidates. The merged text is not allowed to discard a candidate-specific capability merely to simplify the entry.

Because merging can change individually validated content, a multi-candidate consolidation must pass regression certification. Let Q_g contain all tasks that contributed candidates to consolidated target g. The same consolidated content and hash are installed for every replay, and each $q \in Q_g$ is executed from its standard initial environment in a fresh, history-free session with its first-exploration routing fixed. The initial full replay over Q_g does not consume the reflection budget.

If every replay succeeds, the target is certified. Otherwise, a reflection agent receives the initial entry, all accepted candidates, the current consolidation, and every regression prompt, raw trajectory, evaluator outcome, and screenshot. It chooses between two actions:

1. **Revise.** If the consolidated content caused or contributed to any failure, a complete replacement is regenerated from the initial entry and all accepted candidates rather than edited cumulatively from the failed consolidation. Previously passed results are invalidated, and all tasks in Q_g are replayed.

2. **Keep.** The current consolidation may be retained only when every current failure is attributable to grounding and the content itself requires no change. Passed tasks remain certified, and only the failed tasks are replayed.

Each reflection followed by its required replay consumes one of at most $M_{\text{consolidation}}$ cycles. Infrastructure or API failures are handled separately and are not treated as task failures. If certification still fails after the allowance is exhausted, target g is rejected: a revised target retains its initial version, while an uncertified addition is omitted. After all target groups are resolved, $\mathcal{L}_{\text{final}} = \text{Consolidate}(\mathcal{L}_0, \mathcal{P})$ is frozen and evaluated on V with standard initial environments, fresh sessions, normal routing, and no evolution history. No additional global regression is performed.

I.6 Stopping Conditions and Budgets

The default configuration uses

$$M_{\text{explore}} = 3, \qquad M_{\text{submit}} = 3, \qquad M_{\text{consolidation}} = 3. \tag{6}$$

Only task explorations consume M_{explore}; their diagnosis, candidate writing, and one independent candidate validation do not. Only submission validations consume M_{submit}. The initial regression of a multi-candidate consolidation does not consume $M_{\text{consolidation}}$; each subsequent reflection-and-replay cycle does.

Excluding consolidation replay, a conservative execution bound is

$$N_{\text{exec}} \leq |E|\big(2M_{\text{explore}} + M_{\text{submit}}\big), \tag{7}$$

which counts at most one candidate validation after each task exploration and at most M_{submit} submission validations. This is intentionally loose because a task that enters submission cannot also fail all task explorations. For a multi-candidate consolidation group g, regression adds at most $(1 + M_{\text{consolidation}})|Q_g|$ task executions; keep decisions can reduce this by retaining already passed tasks. These bounds exclude final evaluation, diagnosis and writing calls, source retrieval, routing, and separately limited infrastructure retries.

Evolution outcomes and consolidation replays are process measurements and are not substituted for held-out evaluation results.

J Case Study: Complementary Roles of Skills and References in KAlgebra

KAlgebra task C-05 illustrates how procedural guidance and shared expression knowledge support different parts of one task. We compare three executions with autonomous routing: Skills and References together, Skills only, and References only.

J.1 Task, Routing, and Selected Knowledge

Task and initial state. The task instruction is: *"Load script from `~/script.kal` where vector a are defined. Let $b = (\sqrt{3}, 2)$; calculate their inner product in KAlgebra and store the result in variable `p`."* The script contains `a := vector{-1, root(3, 3)}`. Loading it defines $a = (-1, \sqrt[3]{3})$, so the required scalar result is

$$p = a \cdot b = -\sqrt{3} + 2\sqrt[3]{3} \approx 1.15244833305.$$

Initially, a, b, and p are undefined.

Run settings and routing. All runs use Grok-4.6 (xhigh), the same initial environment, and five interaction turns; a turn may include multiple GUI actions or expression submissions. The model autonomously routes over both resource types in the joint condition. Each ablation disables one type while retaining autonomous routing over the other. The selected entries are listed below; the Skill-only run selects two additional Skills beyond S1.

Condition	Autonomously selected resources
Skill + Reference	S1: script workflow; R1: function catalog; R2: expression syntax
Skill only	S1: script workflow; S2: execution mode; S3: latest-result reuse
Reference only	R1: function catalog; R2: expression syntax

S1: Save and rerun Calculator instruction sequences. The relevant procedural instruction is:

> "To execute the saved sequence later, choose Calculator → Load Script... (Ctrl+L) and select the script file. This executes the file's instructions sequentially."

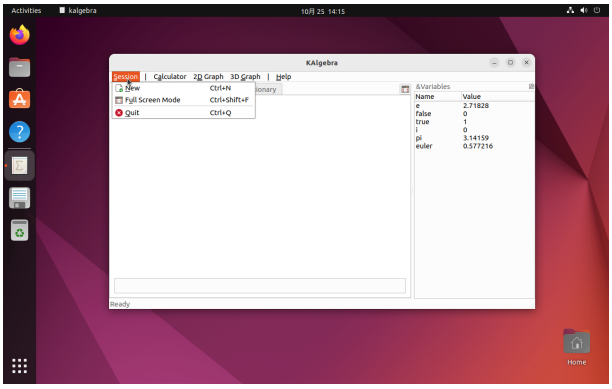
R1: KAlgebra Built-in Function Catalog. The catalog specifies `root(par1, par2)`, illustrated by `x->root(x, 2)`. It also lists the lowercase function name `scalarproduct`. These entries supply the application-specific names and call forms used by the joint run.

R2: KAlgebra Expression Syntax and Operators. This Reference covers function calls, arithmetic, and variable definitions, including `:=` for assignment. Yet Skill-only uses `:=` correctly although none of its three selected Skills explains this syntax. The model already uses it without R2; these trajectories therefore do not establish an independent incremental contribution from R2.

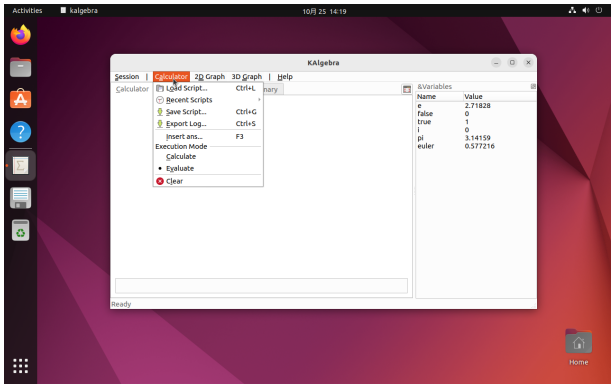
Additional Skills in the Skill-only run. S2, *Choose KAlgebra's calculator execution mode*, explains selecting Calculator → Evaluate or Calculate before entering an expression. S3, *Reuse the latest Calculator result with ans*, describes reusing the latest result through `ans`. Neither supplies the function catalog used by the joint run.

J.2 Single-Resource Trajectories: Different Execution Bottlenecks

Reference only: the loading workflow remains unresolved within the budget. The selected References provide expression knowledge but omit the script-loading workflow. The model first searches Session, then tries Calculator and Ctrl+O. Only after reopening Calculator does it locate Load Script. Its last turn clicks this command without selecting or loading the file, leaving a, b, and p undefined.

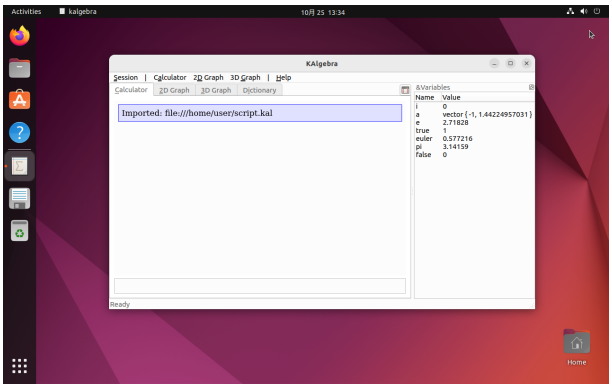


After turn 1. Session has no script-loading command. The model next checks Calculator to find the loading entry.

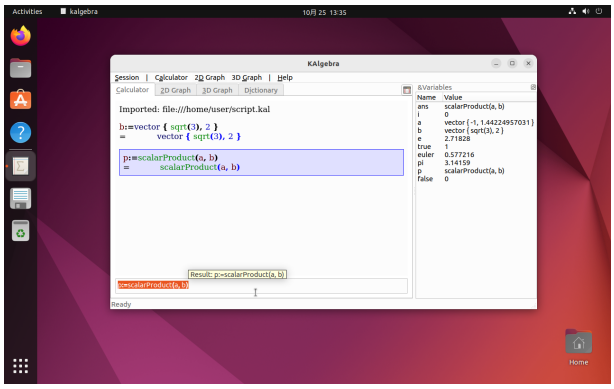


After turn 4. Load Script is now visible. The model selects it in the last turn, but does not select or load the script within the budget.

Skill only: loading succeeds, but the expressions remain symbolic. With S1 available, the model follows Calculator → Load Script and loads a by turn 3. It then enters `b:=vector{sqrt(3), 2}` and `p:=scalarProduct(a, b)`. Both `sqrt(3)` and `scalarProduct(a, b)` remain symbolic in the observed state. The model attributes this to Evaluate mode and attempts to select Calculate in its last turn, without resubmitting the expression. The final p is still `scalarProduct(a, b)`, so evaluation fails.



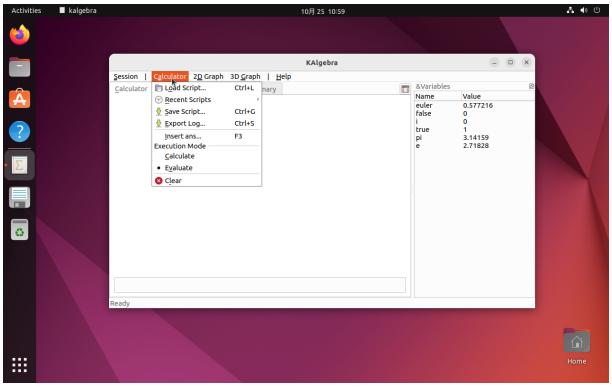
After turn 3. The script has loaded and a is available. The model next tries to define b with `sqrt(3)` and compute p with `scalarProduct(a, b)`.



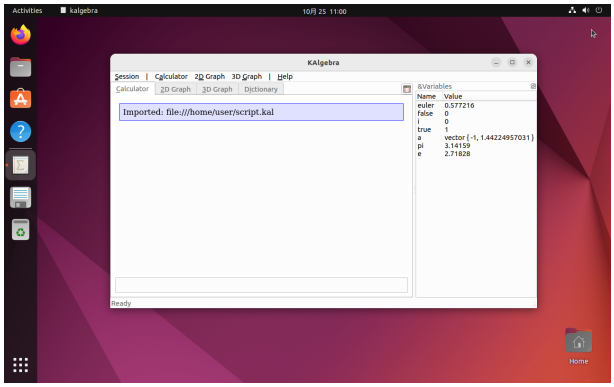
After turn 4. The expressions remain symbolic. The model tries to address this by switching execution mode, rather than correcting the function calls.

J.3 Joint Trajectory: Combining the Workflow and Expression Knowledge

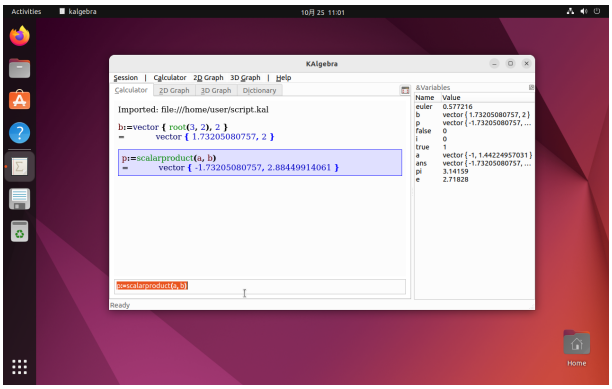
The joint run selects S1, R1, and R2. Its script-loading steps follow S1, and the script is loaded by turn 3. In turn 4, it enters `b:=vector{root(3, 2), 2}`, producing $b = (1.73205080757, 2)$, and attempts the inner product with `p:=scalarproduct(a, b)`. The numerical square root and lowercase function name match R1's function catalog.



After turn 1. Calculator exposes the Load Script command documented by S1. The model selects it to open the file-selection dialog.



After turn 3. The script has loaded and a is available. The model next uses `root(3, 2)` and `scalarproduct` to define b and attempt the inner product.



Feedback correction.

After turn 4. b is numerical, but $p = (-1.73205080757, 2.88449914061)$ is a vector. The model recognizes the mismatch and, in turn 5, sums component products:

`p:=a[1]*b[1]+a[2]*b[2]`.

The resulting $p = 1.15244833305$ passes evaluation. This correction follows execution feedback; the References do not explicitly prescribe it. The screenshot precedes this final correction.

Final outcomes. Values below are recorded after turn 5; screenshots show earlier observations.

Condition	Final state	Evaluation
Reference only	a, b, and p are undefined	Fail
Skill only	`b=vector{sqrt(3),2}`; `p=scalarProduct(a,b)`	Fail
Skill + Reference	$b = (1.73205080757, 2)$; $p = 1.15244833305$	Pass

Complementary contributions. The single-resource runs encounter different bottlenecks: completing script loading and obtaining the required numerical result. The observable complementarity in the joint run lies primarily in S1's loading procedure and R1's application-specific function knowledge, followed by execution feedback for the final correction. These trajectories are consistent with the aggregate ablation results in Section 5.2.

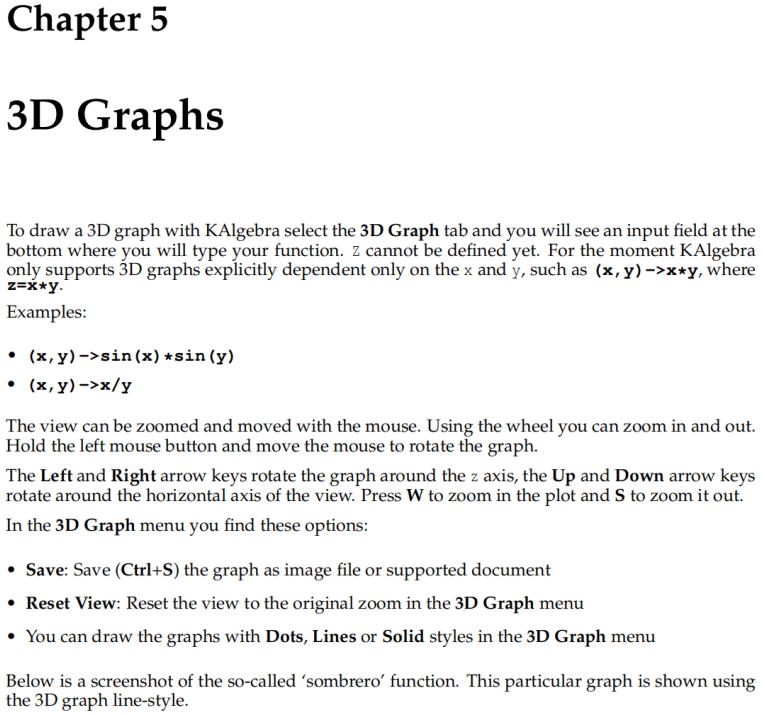
K Case Study: How Self-Evolve Repairs Resource–Environment Mismatches and Improves Task Performance

We use KAlgebra surface-plotting tasks to examine how a mismatch between available documentation and the deployed software affects the initial Skills and References, and how Self-Evolve addresses it through execution feedback. Tasks B-14 and C-19 in Expanded–Verified expose the initial failures and support Skill revision. We then examine whether the frozen updated Skill helps previously failed tasks in the disjoint Verified set succeed. All runs use Grok 4.6 (xhigh).

K.1 Resource–Environment Mismatch: Why Self-Evolve Is Needed

Resource-to-Skill organizes documented procedures into callable Skills and References, but their applicability depends on whether the documentation describes the deployed environment. During resource collection, we could not retrieve documentation matching the deployed software version and used the relevant documentation available to us. The resulting resources therefore describe a different software version.

The collected KAlgebra handbook states: *"Z cannot be defined yet. For the moment KAlgebra only supports 3D graphs explicitly dependent only on the x and y."* The initial Skill, *Plot an explicit x-and-y 3D surface*, incorporates this restriction as a precondition, while the associated 3D Graph Reference retains the documented guidance. As the later executions demonstrate, the deployed software also accepts three-variable mappings and implicit equations involving z.



Chapter 5

3D Graphs

To draw a 3D graph with KAlgebra select the **3D Graph** tab and you will see an input field at the bottom where you will type your function. `z` cannot be defined yet. For the moment KAlgebra only supports 3D graphs explicitly dependent only on the `x` and `y`, such as `(x,y)->x*y`, where `z=x*y`.

Examples:

- `(x,y)->sin(x)*sin(y)`
- `(x,y)->x/y`

The view can be zoomed and moved with the mouse. Using the wheel you can zoom in and out. Hold the left mouse button and move the mouse to rotate the graph.

The **Left** and **Right** arrow keys rotate the graph around the `z` axis, the **Up** and **Down** arrow keys rotate around the horizontal axis of the view. Press **W** to zoom in the plot and **S** to zoom it out.

In the **3D Graph** menu you find these options:

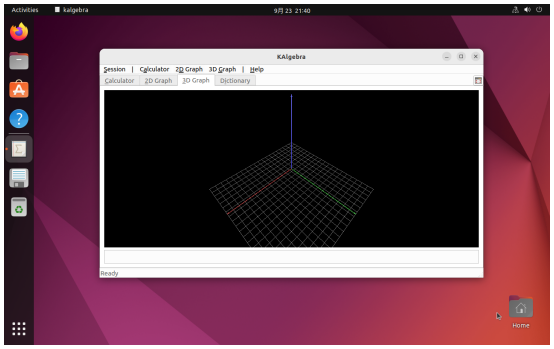
- **Save**: Save (**Ctrl**+**S**) the graph as image file or supported document
- **Reset View**: Reset the view to the original zoom in the **3D Graph** menu
- You can draw the graphs with **Dots**, **Lines** or **Solid** styles in the **3D Graph** menu

Below is a screenshot of the so-called 'sombrero' function. This particular graph is shown using the 3D graph line-style.

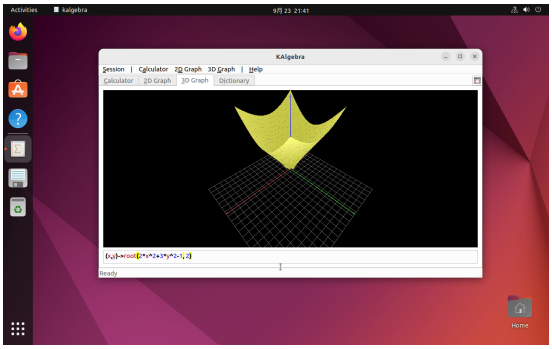
Source documentation. The collected handbook restricts 3D plotting to explicit dependence on x and y. This restriction is inherited by the initial resources but does not describe the deployed environment's full input capabilities.

B-14: initial execution under the routed resources. The task is to plot $2x^2 + 3y^2 - z^2 = 1$. The model initially routes to the 3D-plotting Skill and two References: the function catalog

and the 3D Graph reference. Following the explicit-only guidance, the first exploration enters `(x,y)->root(2*x^2+3*y^2-1,2)`. This selects only the positive root and plots one branch; the task fails. This initial exploration uses no failure history or revised Skill.

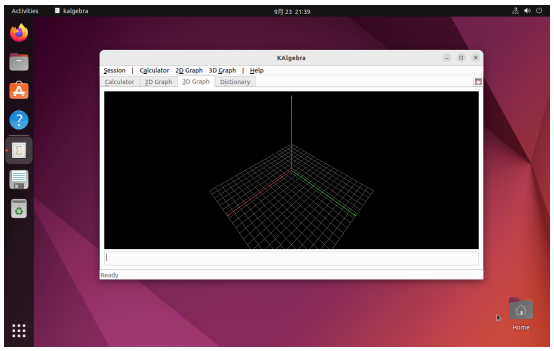


B-14, initial state. The 3D Graph input is empty.

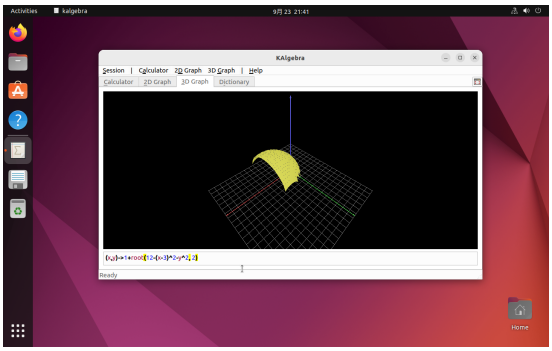


After entering the root mapping. Only one branch is plotted; evaluation fails.

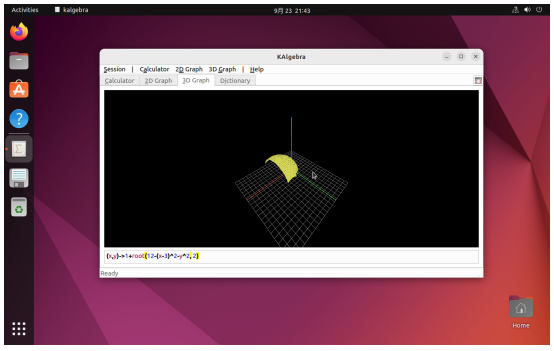
C-19: initial execution under the routed resources. The task is to plot a sphere with diameter endpoints $A(1,-2,3)$ and $B(5,2,-1)$. Its center is $(3,0,1)$ and its squared radius is 12. The initial Skill and routed 3D Graph Reference likewise direct the first exploration toward an explicit mapping. The positive-root expression produces only the upper hemisphere. Zooming out does not change the incomplete surface, and the task fails. This exploration also precedes any failure-history input or Skill revision.



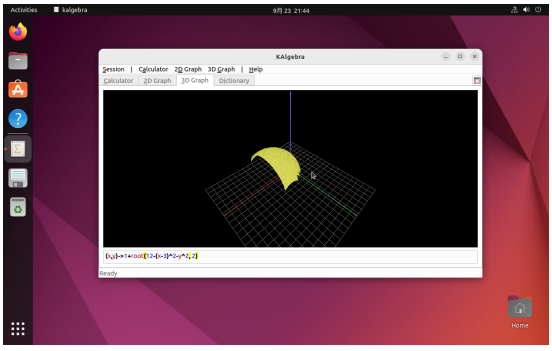
C-19, initial state. The 3D Graph is empty.



After entering the positive-root mapping. Only the upper hemisphere is plotted.



After zooming out. The sphere remains incomplete.



End-of-exploration checkpoint. The incomplete sphere remains; evaluation fails.

Implication for the initial resources. Both trajectories show how the documented restriction narrows the agent's solution path to explicit $z = f(x, y)$ expressions. For these tasks, a single root

branch covers only part of the required surface. An accepted expression and a visible plot therefore do not establish task completion. These failures motivate using actual execution to test and revise the inherited procedural constraints.

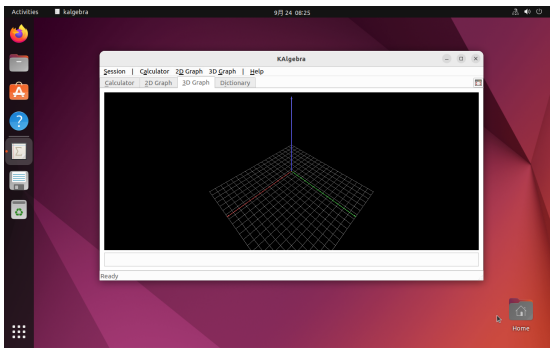
K.2 From Execution Failure to Skill Revision

B-14 and C-19 belong to the Expanded–Verified evolution set. Throughout exploration, the initial library and each task's first routed resource set remain fixed. After failure, the system retains the execution trajectory, screenshots, evaluator result, diagnosis, and candidate-validation records. Later explorations receive the accumulated history for that task, with historical executions explicitly identified and their mapped screenshots included in the prompt. They can therefore reconsider unsuccessful actions while still starting from the original routed resources.

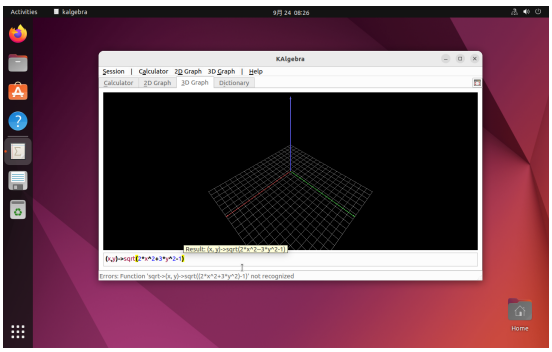
Candidate revisions are tested separately in fresh environments without exploration history. After a successful exploration, the submission step uses that trajectory and the earlier failures to revise the original Skill, followed by another independent validation. The discovered procedure must thus be written into the Skill and reproduce success without access to prior trajectories or diagnoses. The following sections continue from the initial failures shown above.

K.3 B-14: Revising the Single-Branch Procedure

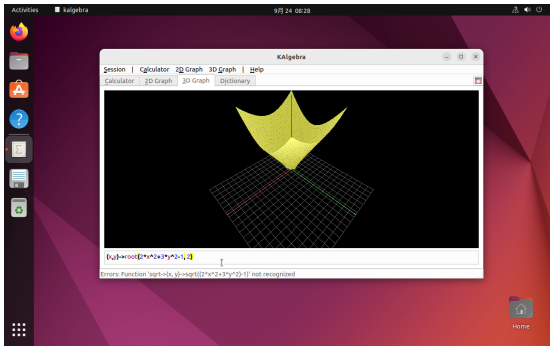
First candidate: changing the square-root expression. After the initial failure, a candidate proposes `sqrt(...)`. Independent validation reports that the function is not recognized. Falling back to `root(...)` again produces one branch, so the candidate fails.



Candidate validation 1, initial state. The proposed revision is tested without history.



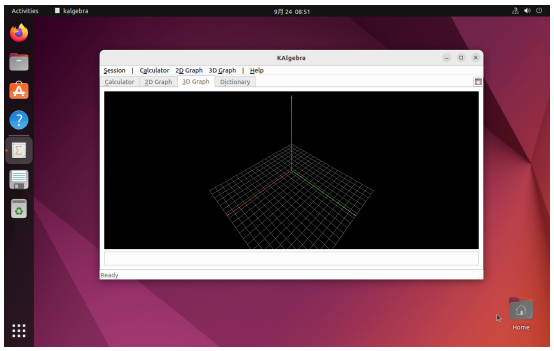
After entering `sqrt(...)`. The function is not recognized.



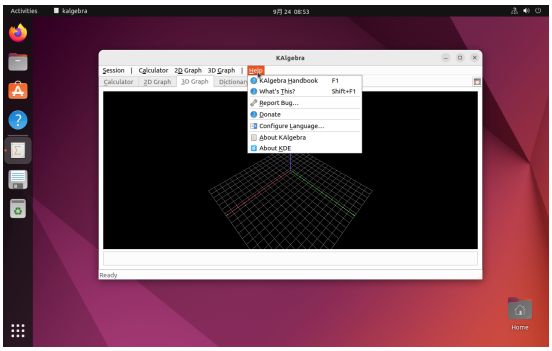
After fallback to `root(...)`. Only one branch appears; validation fails.

Second exploration: using the first failure records. Exploration 2 receives exploration 1 and candidate validation 1: two executions with five screenshots. The agent consults the in-app

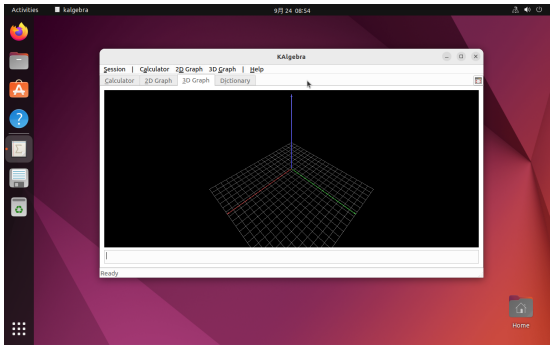
Help for a usable 3D-plotting procedure, but the search does not establish a working solution.



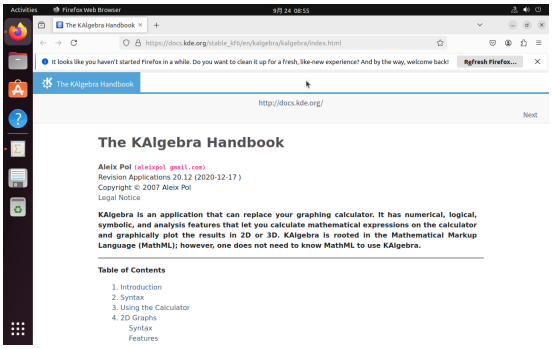
Exploration 2, initial state. Earlier failure records are supplied for this run.



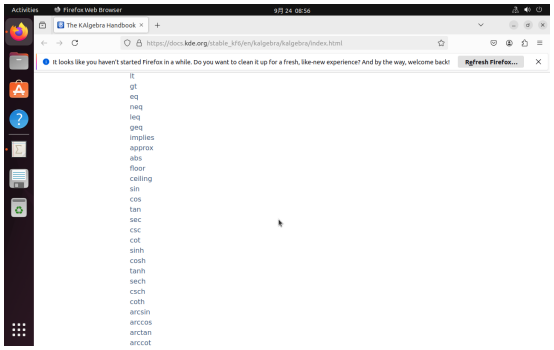
After opening Help. The agent searches for plotting guidance.



After F1. The plotting window remains visible.

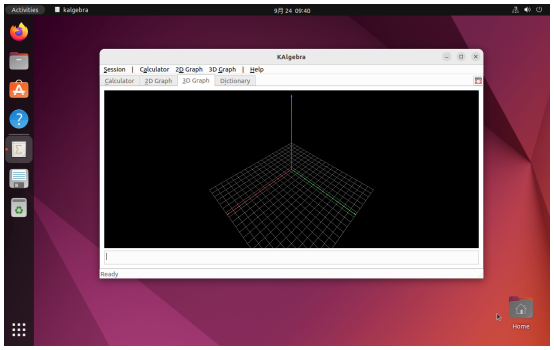


Next Help checkpoint. The handbook is visible in a browser.

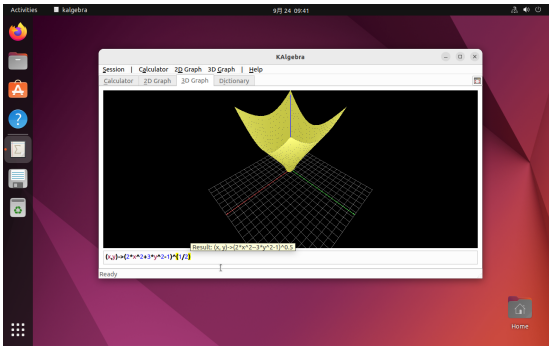


After searching for "3D Graphs." The search does not identify a working full-surface procedure.

A subsequent power-form candidate is also independently tested. Although the environment accepts the expression, it still plots only one branch. Changing the square-root notation has not resolved the incomplete geometry.

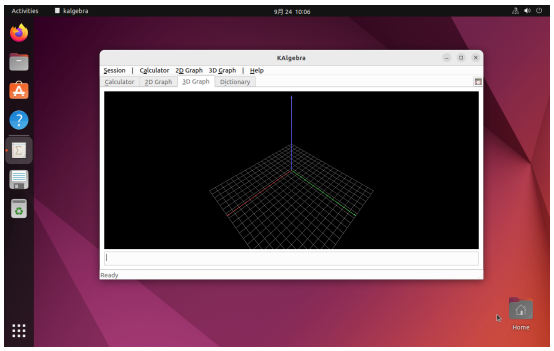


Candidate validation 2, initial state. The power-form revision is tested without history.

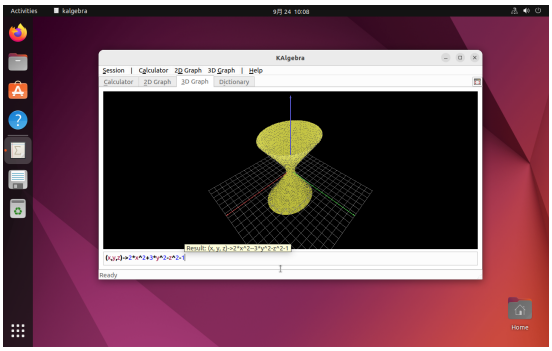


After entering the power mapping. The expression is accepted, but the surface remains incomplete.

Third exploration: using the accumulated evidence. Exploration 3 receives both earlier explorations and both candidate validations: four executions with twelve screenshots. The agent then enters `(x,y,z)->2*x^2+3*y^2-z^2-1`. The environment accepts this three-variable mapping and renders the complete one-sheet hyperboloid; evaluation passes. This execution directly contradicts the initial explicit-only restriction.

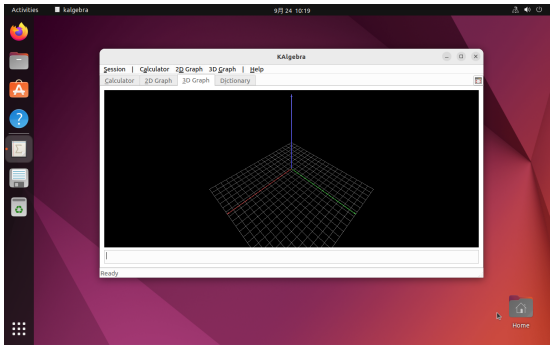


Exploration 3, initial state. The accumulated history accompanies the original routed resources.

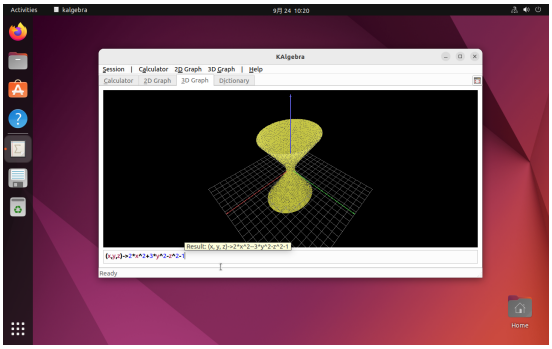


After entering the three-variable mapping. The complete hyperboloid appears; evaluation passes.

Encoding and independently validating the update. The submitted revision removes the incorrect restriction on defining z, retains the explicit-surface workflow, and adds a conditional three-variable mapping path for implicit surfaces. It also explains that a single explicit root branch does not represent the complete surface. A fresh run using the revised Skill reproduces the full hyperboloid without exploration history, showing that the successful procedure has been captured in the Skill.



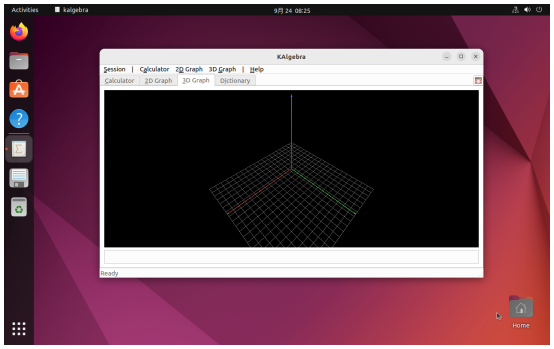
Independent validation, initial state. The revised Skill is loaded without history.



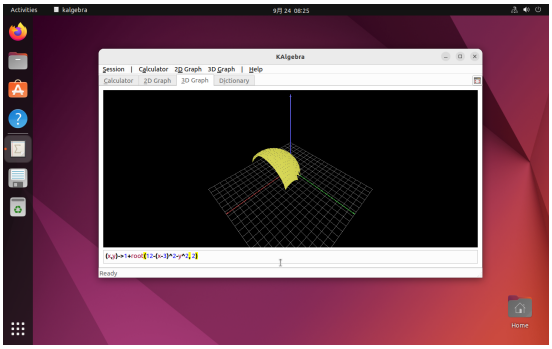
After following the revised mapping. The complete hyperboloid is reproduced; validation passes.

K.4 C-19: Revising the Two-Hemisphere Procedure

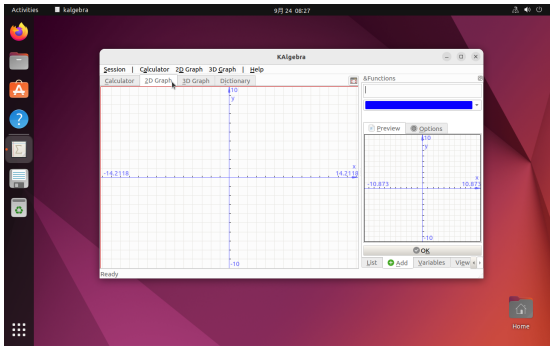
First candidate: combining explicit branches. Following the initial failure in Section K.1, a candidate proposes plotting the positive and negative root branches to form the full sphere. Its independent validation does not produce the required combination: the visible result remains a single hemisphere.



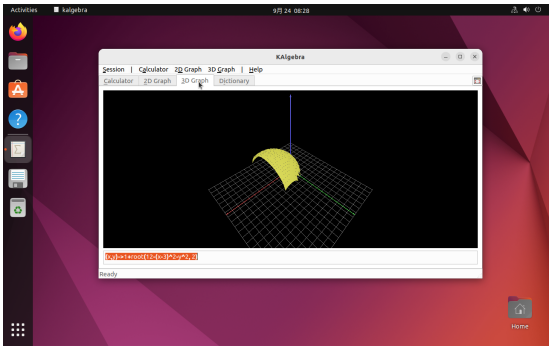
Candidate validation, initial state. The two-branch revision is tested without history.



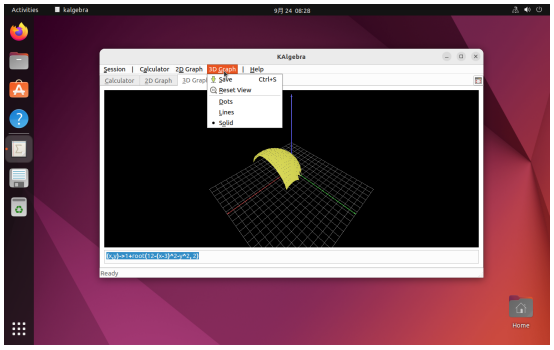
After plotting the positive branch. Only the upper hemisphere is visible.



After switching tabs. A second plotted branch has not been produced.

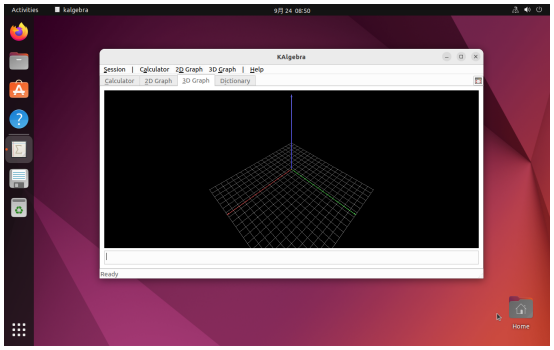


After returning to 3D Graph. The input still contains the positive branch.

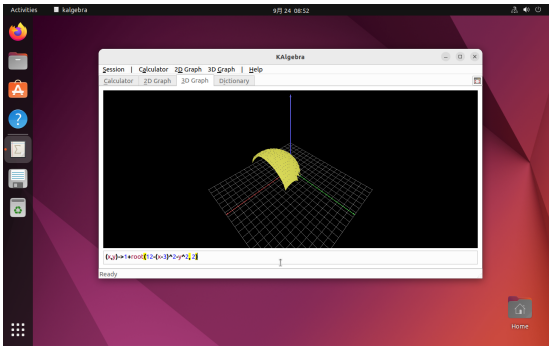


After opening the 3D Graph menu. The sphere remains incomplete; validation fails.

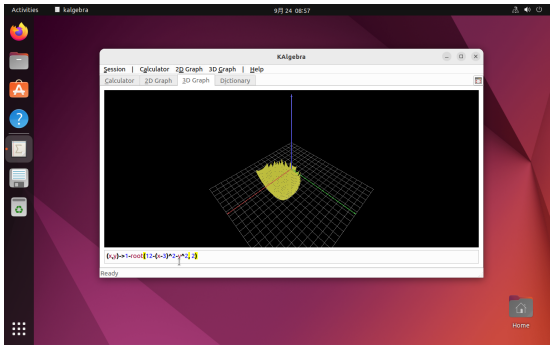
Second exploration: identifying replacement and testing an implicit equation. Exploration 2 receives the initial exploration and failed candidate validation: two executions with nine screenshots. Entering the positive and then the negative root reveals that the latter replaces the former in the input field. The displayed surface changes from the upper to the lower hemisphere rather than combining them. The agent then enters `(x-3)^2+y^2+(z-1)^2=12`. The complete sphere appears and evaluation passes.



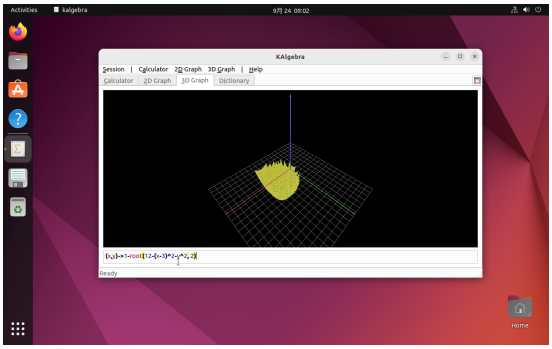
Exploration 2, initial state. The earlier failure records are supplied.



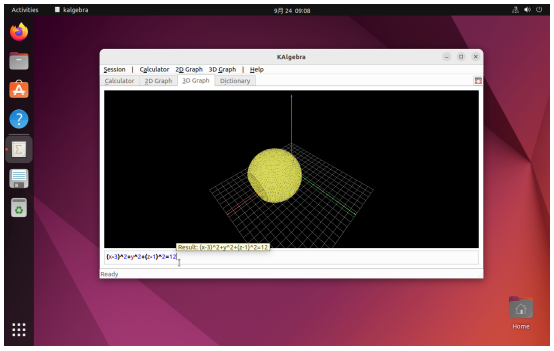
After entering the positive branch. The upper hemisphere appears.



After entering the negative branch. The lower hemisphere replaces the upper one.

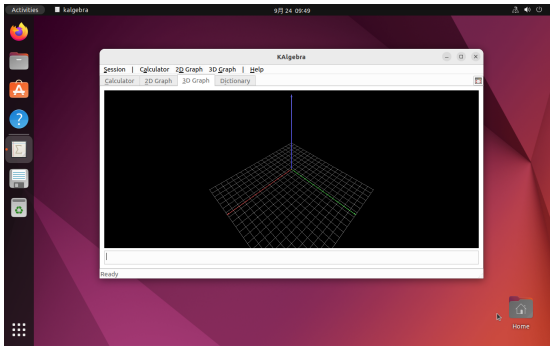


Checkpoint before the implicit equation. Only the lower hemisphere remains.

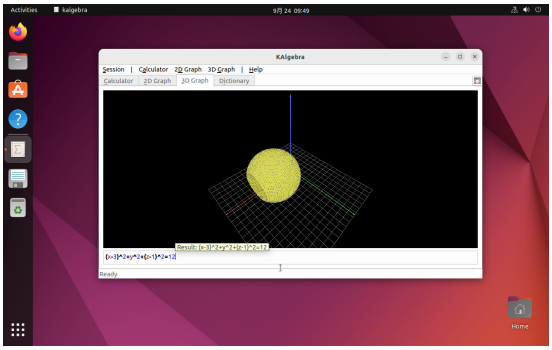


After entering the implicit equation. The complete sphere appears; evaluation passes.

Encoding and independently validating the update. The revision retains the explicit-mapping procedure, corrects the version-dependent restriction, and adds the observed implicit-equation route. It also states that entering the two explicit hemisphere expressions successively replaces the input rather than combining the surfaces. Without exploration history, an independent run using the revised Skill again produces the complete sphere.



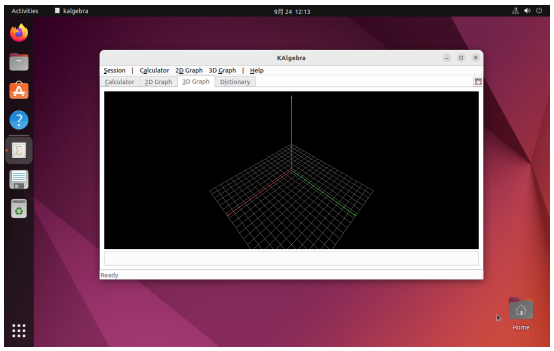
Independent validation, initial state. The revised Skill is loaded without history.



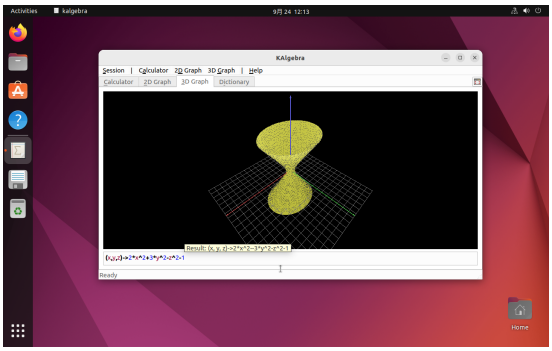
After entering the sphere equation. The complete sphere is reproduced; validation passes.

K.5 Consolidation and Regression: Preserving Both Revisions

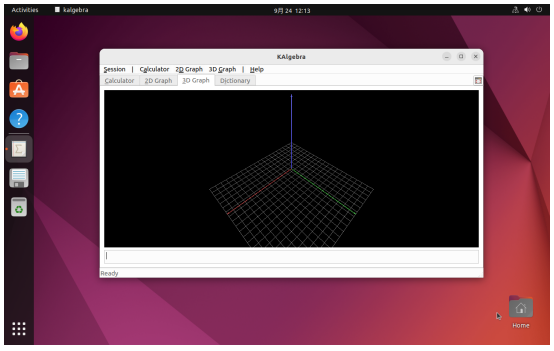
The two accepted candidates revise the same initial Skill. They are consolidated into one entry that preserves the separately demonstrated procedures: the three-variable mapping used in B-14 and the implicit equation used in C-19. The consolidated Skill is replayed on both contributing tasks without history. Both tasks succeed again, showing that consolidation retains the capabilities established by the two revisions.



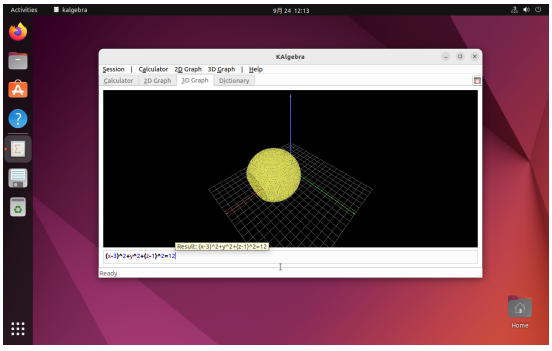
B-14 regression, initial state. The consolidated Skill is tested without history.



B-14 regression result. The complete hyperboloid is reproduced.



C-19 regression, initial state. The consolidated Skill is tested without history.



C-19 regression result. The complete sphere is reproduced.

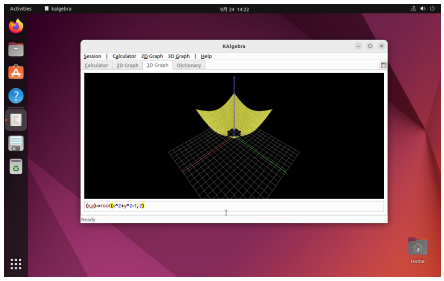
The initial failures have now led to validated, reusable procedures through accumulated history, exploration, revision, and regression. We next test whether these procedures help tasks that did not contribute to the revisions.

K.6 Transfer to Verified: Previously Failed Tasks Succeed

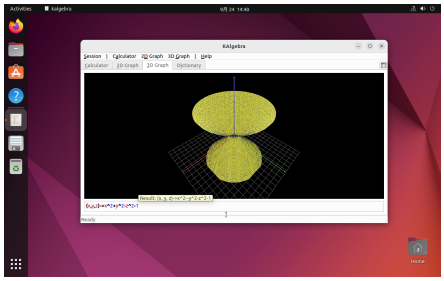
After consolidation, the updated Skill is frozen. Tasks A-03, C-14, and C-17 belong to the disjoint Verified set and did not participate in the revisions above. All three failed with the initial library. Their subsequent runs route to the updated Skill without receiving the evolution histories of B-14 or C-19. The table lists the expressions entered in the successful runs.

Task	Expression with updated Skill	Initial library	Updated Skill
A-03	`(x,y,z)->x^2+y^2-z^2-1`	Failed	Passed
C-14	`x^2+y^2+z^2=2`	Failed	Passed
C-17	`(x,y,z)->(x+z)^2+y^2-1`	Failed	Passed

A-03. The initial attempt renders only one branch. With the updated Skill, the agent uses a three-variable mapping to render the complete hyperboloid and passes evaluation.

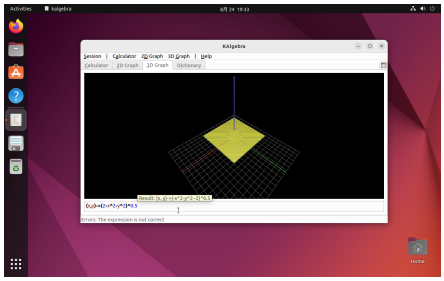


Initial library. The single-branch result is incomplete.

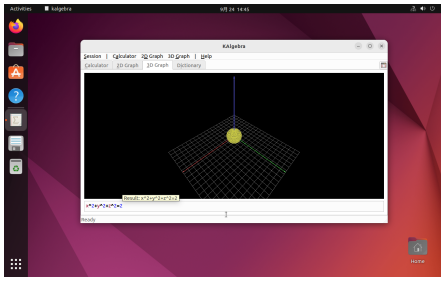


Updated Skill. The complete hyperboloid is rendered; evaluation passes.

C-14. The initial explicit-branch attempt does not form the requested sphere. The updated Skill supports an implicit equation that produces the complete sphere.

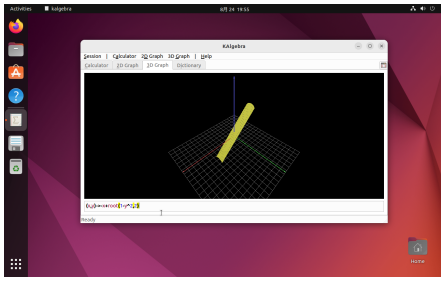


Initial library. The requested complete sphere is not produced.

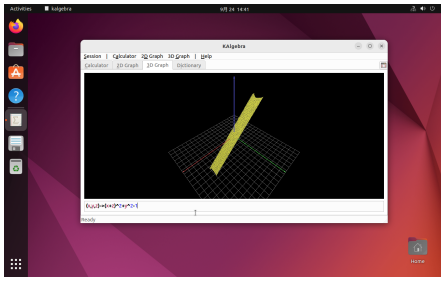


Updated Skill. The implicit equation produces the sphere; evaluation passes.

C-17. The initial explicit-branch construction is incomplete. With the updated Skill, a three-variable mapping produces the required cylinder.



Initial library. The explicit-branch construction is incomplete.

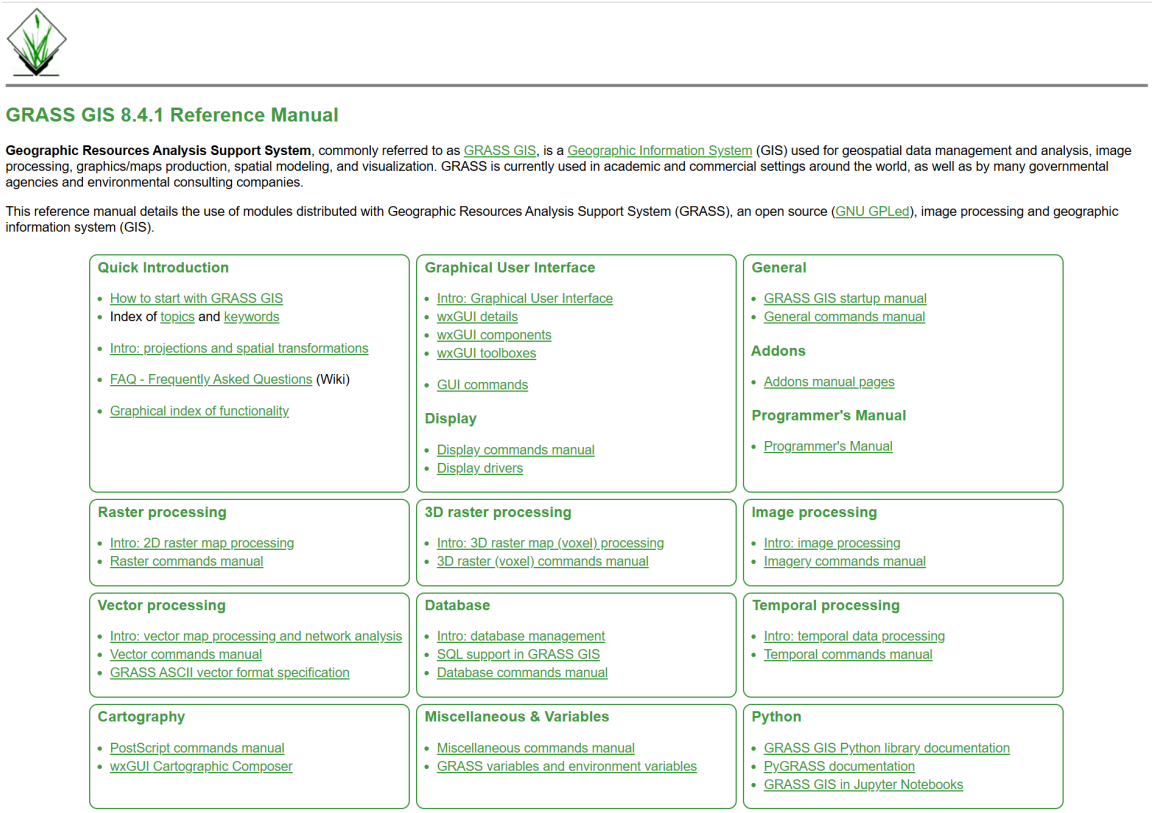


Updated Skill. The required cylinder is produced; evaluation passes.

These cases connect the resource mismatch to improvements on independent tasks. Outdated constraints first manifest as failures on Expanded–Verified tasks. Self-Evolve uses the failure history to discover procedures supported by the deployed environment and incorporates them into independently validated Skills. The updated Skill then helps three previously failed Verified tasks succeed, demonstrating both the need to revise initial resources through execution and the transfer of the resulting procedural knowledge.

L Case Study: Adding a Module-Level Reference for GRASS GIS

Why broad resource coverage leaves a task-relevant gap. The GRASS GIS Reference Manual organizes software knowledge into twelve major sections. The benchmark tasks, however, are concentrated in a much smaller part of this space, primarily the Graphical User Interface (including Display), General, 3D raster processing, and Database sections. Resource-to-Skill runs before the downstream task distribution is known, so its initial construction aims to cover the software broadly. This coverage-oriented strategy can produce Skills and References for material that benchmark tasks rarely use, while leaving frequently needed leaf-level knowledge too general to execute. The `d.vect` command illustrates this mismatch: the initial Display catalog identifies the command but does not provide the syntax and parameter combinations needed by the recurring vector-styling tasks.



GRASS GIS 8.4.1 Reference Manual

Geographic Resources Analysis Support System, commonly referred to as GRASS GIS, is a Geographic Information System (GIS) used for geospatial data management and analysis, image processing, graphics/maps production, spatial modeling, and visualization. GRASS is currently used in academic and commercial settings around the world, as well as by many governmental agencies and environmental consulting companies.

This reference manual details the use of modules distributed with Geographic Resources Analysis Support System (GRASS), an open source (GNU GPLed), image processing and geographic information system (GIS).

Quick Introduction
- How to start with GRASS GIS
- Index of topics and keywords
- Intro: projections and spatial transformations
- FAQ - Frequently Asked Questions (Wiki)
- Graphical index of functionality

Graphical User Interface
- Intro: Graphical User Interface
- wxGUI details
- wxGUI components
- wxGUI toolboxes
- GUI commands

Display
- Display commands manual
- Display drivers

General
- GRASS GIS startup manual
- General commands manual

Addons
- Addons manual pages

Programmer's Manual
- Programmer's Manual

Raster processing
- Intro: 2D raster map processing
- Raster commands manual

3D raster processing
- Intro: 3D raster map (voxel) processing
- 3D raster (voxel) commands manual

Image processing
- Intro: image processing
- Imagery commands manual

Vector processing
- Intro: vector map processing and network analysis
- Vector commands manual
- GRASS ASCII vector format specification

Database
- Intro: database management
- SQL support in GRASS GIS
- Database commands manual

Temporal processing
- Intro: temporal data processing
- Temporal commands manual

Cartography
- PostScript commands manual
- wxGUI Cartographic Composer

Miscellaneous & Variables
- Miscellaneous commands manual
- GRASS variables and environment variables

Python
- GRASS GIS Python library documentation
- PyGRASS documentation
- GRASS GIS in Jupyter Notebooks

The manual spans twelve major sections; task demand is concentrated in a subset. The `d.vect` command is documented under Display.

Prior Verified runs expose a recurring gap in this task family. B-04 requires an area fill color, B-06 a line width, and B-08 a point-symbol size. Their initial routed resources contained a broad

Display command catalog, but the corresponding executions did not reach the required `d.vect` parameters. These failures motivate adding knowledge at the module level rather than writing a separate task-specific procedure for each map.

B-18: diagnosing and filling the knowledge gap. B-18 is an Expanded–Verified task in the evolution set: *"Display countries@PERMANENT as red boundary lines only in GRASS GIS."* The initial resource set exposed the broad Display catalog, but not enough executable detail to compose the required command. Grok Build examined the failure evidence and the official GRASS manual, then selected a Reference rather than a Skill. The diagnosis was that the missing capability was declarative module knowledge—command syntax, option semantics, accepted values, defaults, and parameter composition—rather than an unlearned multi-step workflow. Grok Build browsed three pages from the same official manual. All 12 retained source-evidence records passed the later source audit, and the generator did not receive Verified task prompts or answers.

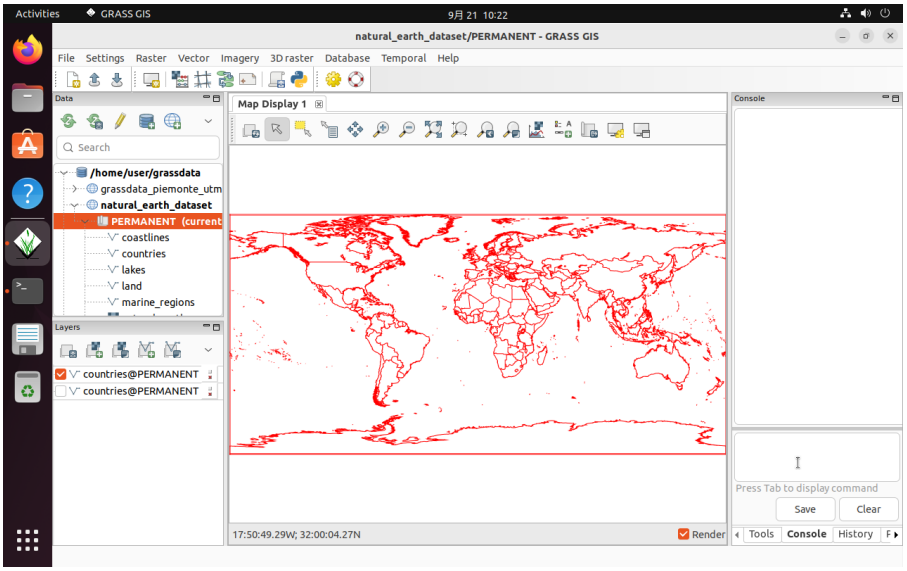
The resulting module-level Reference, *d.vect Vector Display and Styling*, generalizes beyond the B-18 map and color. Its relevant guidance includes:

> `color=name` controls feature color; `fill_color=name` controls area fill. Areas use `fill_color` for their fill and `color` for their outline. Use `fill_color=none` to make areas transparent, or `color=none` to suppress area outlines.
> `width=integer` sets line width (default: 0).
> `size=float` sets point-symbol size (default: 5).
> `attribute_column=name` selects the label column; `font`, `xref`, and related options control label appearance and placement.

This knowledge is directly relevant to several task families: independent outline and fill controls support B-18 and B-04; line-width and symbol-size options support B-06 and B-08; label parameters provide a reusable route for other vector-styling requests. The Reference describes reusable `d.vect` behavior rather than a fixed answer to B-18 in the evolution set: its options can be composed for other maps, colors, and styling requests.

B-18 validation: applying the added module knowledge. During validation, the candidate Reference is force-injected into B-18's resource set. Using the command syntax and independent outline and fill controls provided by the Reference, the agent constructs the command below and executes it in the Command Console. The map is rendered with red boundaries and no area fill, and B-18 passes. **The added Reference supplies the parameter knowledge missing from the initial catalog, enabling the agent to compose the required command.**

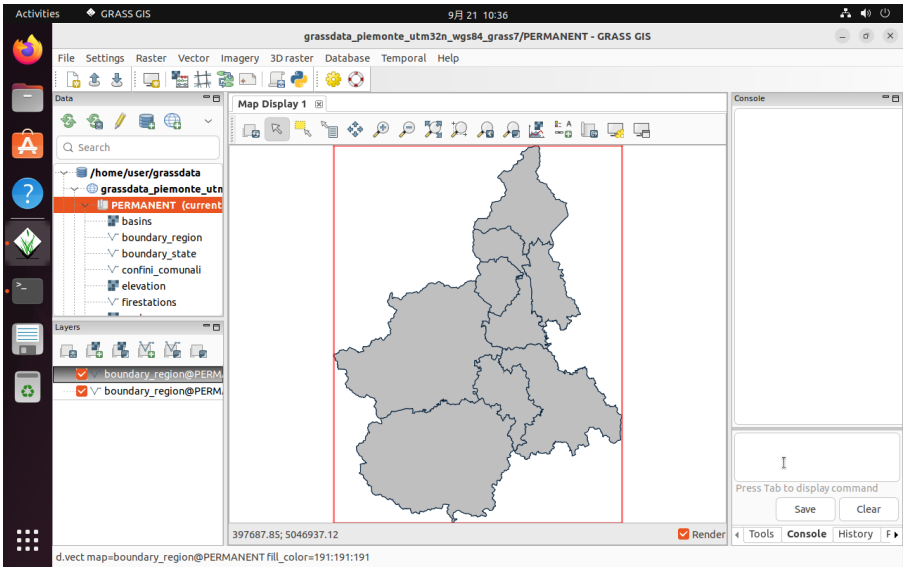
```
d.vect map=countries@PERMANENT type=boundary color=red fill_color=none
```



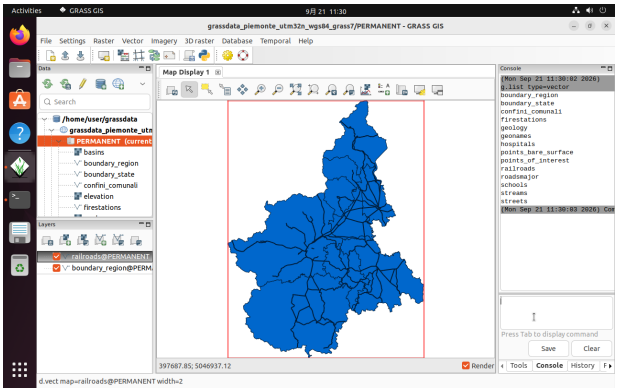
B-18: using the added Reference, the agent renders red boundaries with no area fill.

Transfer to Verified: three related styling tasks. B-04, B-06, and B-08 all failed in earlier Verified runs with the initial library. After validation, the new `d.vect` Reference was made available in the evolved library. In the subsequent Verified evaluation, resource selection was free: each task routed to the new Reference. The successful runs used three different parameter families:

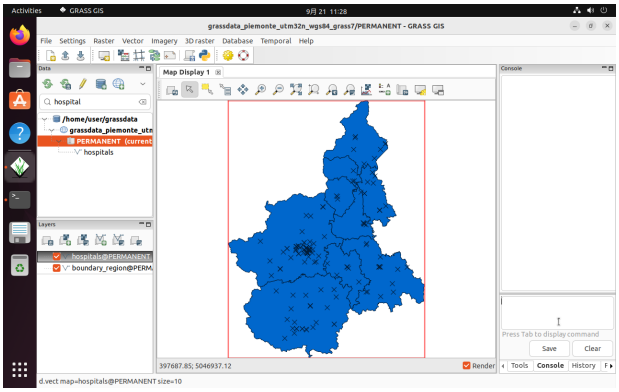
Task	Initial gap	Command in a successful transfer run
B-04	Could not set area fill color	`d.vect map=boundary_region fill_color=191:191:191`
B-06	Did not set `width=2`	`d.vect map=railroads width=2`
B-08	Did not set `size=10`	`d.vect map=hospitals size=10`



B-04: the area is gray after applying `fill_color=191:191:191`.



B-06: the Console executes `width=2`.



B-08: the Console executes `size=10`.

Takeaway. This case shows how Self-Evolve addresses the mismatch between the broad scope of source resources and the specific knowledge required by downstream tasks through knowledge addition. The initial library covered GRASS GIS broadly but omitted the `d.vect` details required by these tasks. Execution feedback from the evolution set guided Grok Build to add a reusable module-level Reference. **The added knowledge filled this task-relevant gap, enabling success on B-18 and related Verified tasks that the initial library had failed to support.**

M Case Study: Outdated Interface Guidance in Celestia

Task A-05 asks the agent to display dwarf-planet orbits but not moon orbits in Celestia. We examine an initial-library run by Grok-4.6 (xhigh), before any Skill or Reference evolution. The first-stage router selected the Skill *Control rendering features and display quality with keyboard shortcuts* and the Reference *Celestia label, info-text, constellation, and star-filter controls*. The source User's Guide describes Celestia 1.6.1, whereas the target environment runs Celestia 1.7.0 with a Qt interface.

From source guidance to the target interface. The selected Reference places the checkboxes for orbit categories in the lower part of a *View Options* window and also mentions the `O` key for orbit paths. The selected Skill condenses this guidance to pressing `O` to toggle planet orbits; it does not explain how to choose dwarf-planet or moon orbits separately in the target interface. Figure 7 shows the old *View Options* window in the source guide. In the deployed Qt interface, neither the Display nor the View menu contains this window. Instead, the toolbar's `O` dropdown exposes separate checkboxes for *Dwarf Planets* and *Moons* (Figure 8). The target-interface screenshot is an independent illustration of the available control, not a frame from the failed run.

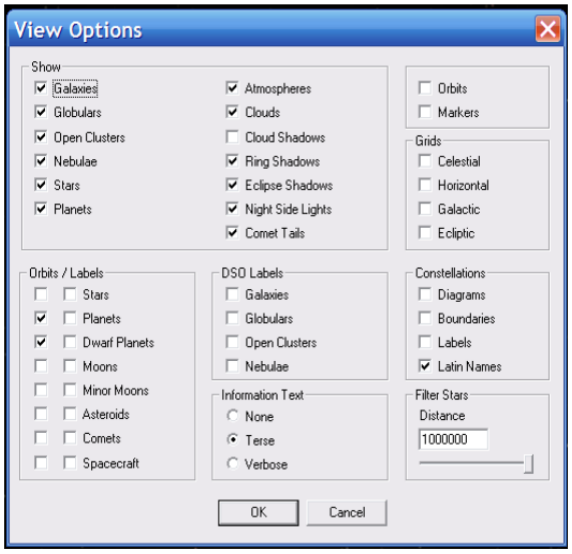


Figure 7: The *View Options* window shown in the collected Celestia 1.6.1 User's Guide. The selected Reference retains this old location for orbit-category controls.

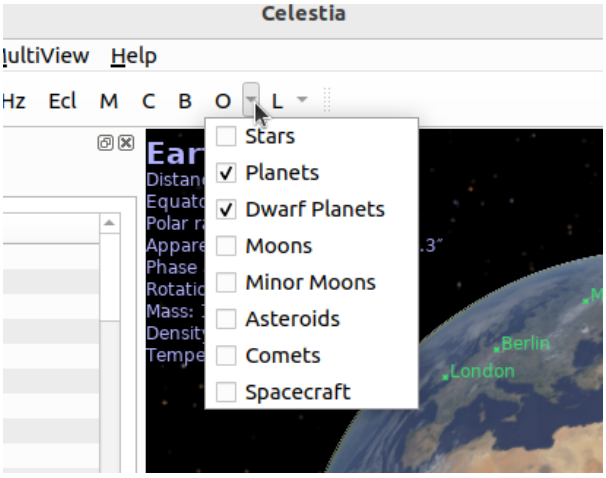
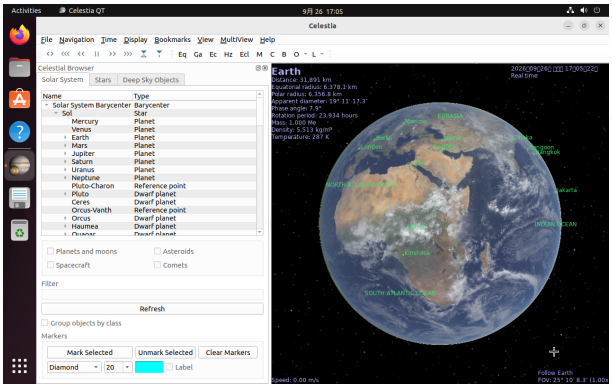
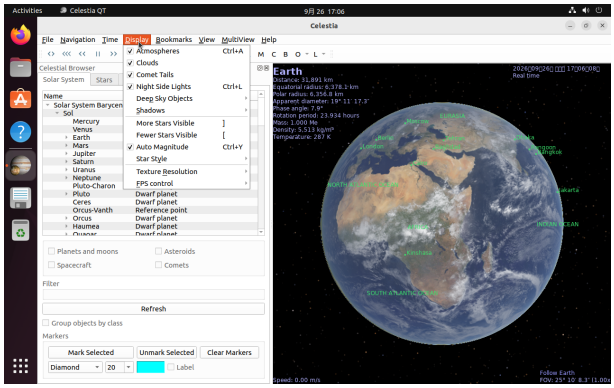


Figure 8: The target Celestia 1.7.0 Qt interface. The toolbar's `O` dropdown allows dwarf-planet and moon orbits to be selected independently; the screenshot shows *Dwarf Planets* checked and *Moons* unchecked. This image illustrates the available control and is separate from the failed trajectory.

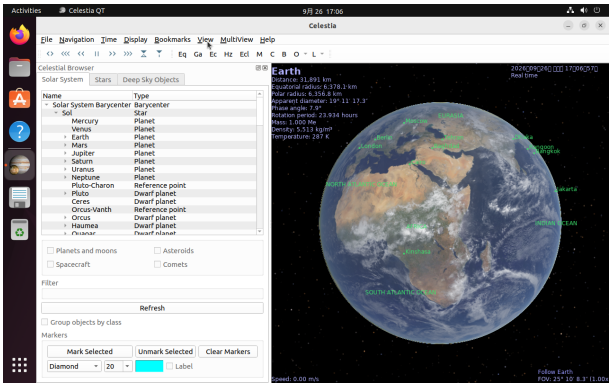
Execution with the initial Skill and Reference. Figure 9 presents every screenshot from the five-action run. Each screenshot is a pre-action observation; its caption describes the action chosen next. Although the toolbar's `O` control is visible in the initial state, the agent first states that Display should contain *View Options* and opens that menu. Finding no such entry, it moves to View, retries View with `Alt+V`, and then selects Guides as another possible location for orbit-path options. That action hides the toolbar. Only in its final response does the agent identify the `O` dropdown, but it attempts to restore the toolbar rather than configure the two orbit categories. The run ends without completing A-05, and the task evaluator reports failure.



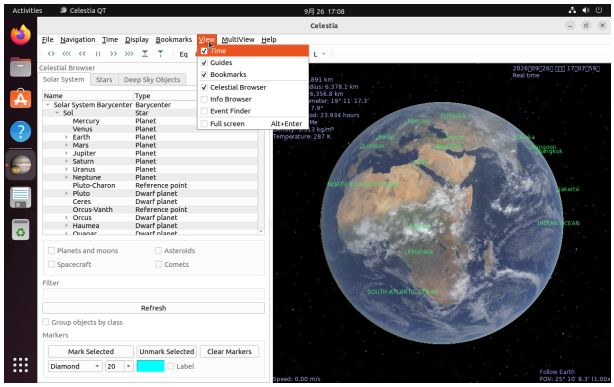
(a) Initial state. The toolbar's `O` control is visible; the agent chooses to open Display.



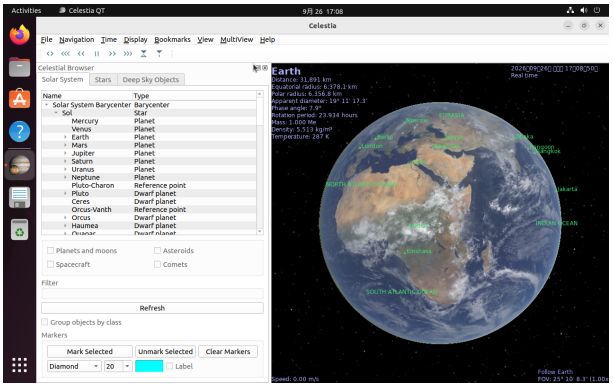
(b) Display menu. No *View Options* appears; the agent turns to View.



(c) After clicking View. The menu is not open; the agent retries with `Alt+V`.



(d) View menu. The old entry is absent here too; the agent selects Guides.



(e) After selecting Guides. The toolbar has disappeared; the agent next tries to restore it.

Figure 9: The complete five-step trajectory of Grok-4.6 (xhigh) with the initial Celestia Skill and Reference on A-05. Panels show observations before the actions described beneath them. The agent spends its actions searching old menu paths and never sets the required orbit categories; evaluation fails.

Implication. This run makes the cost of outdated knowledge concrete. Guidance inherited from the old interface directs the agent toward Display, View, and Guides, consuming its limited action budget even though the target's `O` dropdown is visible from the start. The agent recognizes that control only at the end and does not complete the category-specific orbit settings. Thus, an initial Skill and Reference grounded in documentation can still waste execution steps and lead to task failure when their operational guidance does not match the deployed software.

N Task Requirements and Documentation Scope for Lean 4 and TeXstudio

This appendix compares the Verified task requirements for Lean 4 and TeXstudio with the documentation used to construct their initial Skills and References. Tables 10 and 11 summarize the task goals; the full task instructions are provided with ScienceBoard-VE. The comparison helps explain why additional knowledge from the same software-specific sources provides more limited gains for these applications in Table 3.

N.1 Lean 4: General Proof Guidance and Domain-Specific Theorems

The Lean 4 initial library is constructed from *Theorem Proving in Lean 4* (https://leanprover.github.io/theorem_proving_in_lean4/). As Figure 10 shows, its contents cover general proof methods, including propositions, quantifiers, tactics, induction, and type classes. The Verified tasks in Table 10, by contrast, ask the agent to complete particular formal statements across set theory, algebra, topology, analysis, measure theory, and other mathematical domains. Completing them requires identifying applicable domain definitions and lemmas and assembling proofs that match the given Lean declarations.

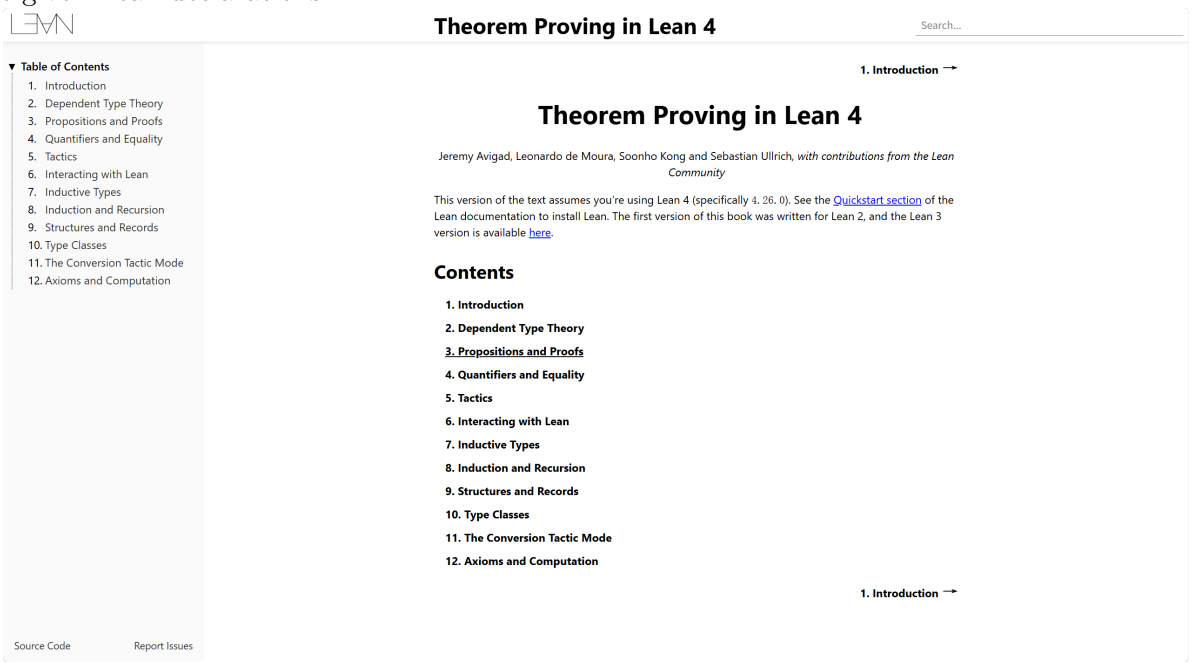


Figure 10: Landing page and table of contents of *Theorem Proving in Lean 4*, the source used for the initial Lean 4 library.

Table 10: Lean 4 Verified tasks and their proof goals. The entries summarize the full formal task statements supplied with ScienceBoard-VE.

Task	Proof goal
A-01	Derive reflexivity from symmetry, transitivity, and an existence condition for a relation.
A-02	Prove a recurrence inequality for the Ackermann function.
A-03	Relate transitivity of a ZF set to powerset and union conditions.
A-04	Prove a cofinality property of a limit ordinal.
A-05	Prove a statement involving cardinal operations.

Table 10 (continued)

Task	Proof goal
B-01	Prove a modular congruence under primality and coprimality assumptions.
B-03	Prove an embedding statement for isomorphic structures.
B-04	Establish a property of many-one equivalence under `Sum.elim`.
B-05	Prove that a quotient of a cyclic group is cyclic.
B-06	Prove a statement about an ideal and a quotient division ring.
B-07	Prove a statement about a normal field extension and an intermediate field.
C-01	Prove that metrizability is preserved under homeomorphism.
C-02	Establish closedness from a derived-set inclusion condition.
C-03	Prove connectedness of the continuous image of a connected set.
C-04	Relate openness conditions between topologies to the T_0 and T_1 properties.
C-05	Prove compactness of a closed subset of a compact set.
D-01	Derive a quantitative bound from a function limit.
D-02	Prove uniform continuity under a bounded-derivative condition.
D-03	Prove that a continuous function vanishes when the integral of its square is zero.
D-04	Establish the cardinality of a countably generated set algebra.
D-05	Construct a measure from the limit of an increasing sequence of measures.
D-06	Establish convergence in measure after composition with a continuous function.
D-07	Prove a limit statement involving a random variable's moment and tail probability.
E-01	Construct a decidability instance for representation as a sum of four integer cubes.
E-02	Construct a decidability instance for an extremal set-family property.
E-03	Prove an edge-count lower bound under a Hamiltonian-walk condition.

The source supplies general Lean proof and interaction knowledge, but it cannot directly enumerate the domain-specific lemmas and proof sequences required by all of these statements. Execution feedback can expose where an attempted proof fails; when the original source does not contain the missing mathematical knowledge, source-grounded additions to the library have limited scope. This task–source gap helps explain why Lean 4 retains a relatively low success rate after self-evolution.

N.2 TeXstudio: Editor Operations and Cross-Application Information

The TeXstudio initial library is constructed from the TeXstudio manual (https://texstudio-org.github.io/). Its contents cover document editing, compilation, PDF viewing, and editor configuration (Figure 11). Table 11 shows that some Verified tasks require only TeX editing or compilation, while others first require a measurement, object property, computation, or bibliographic record from another application or website.

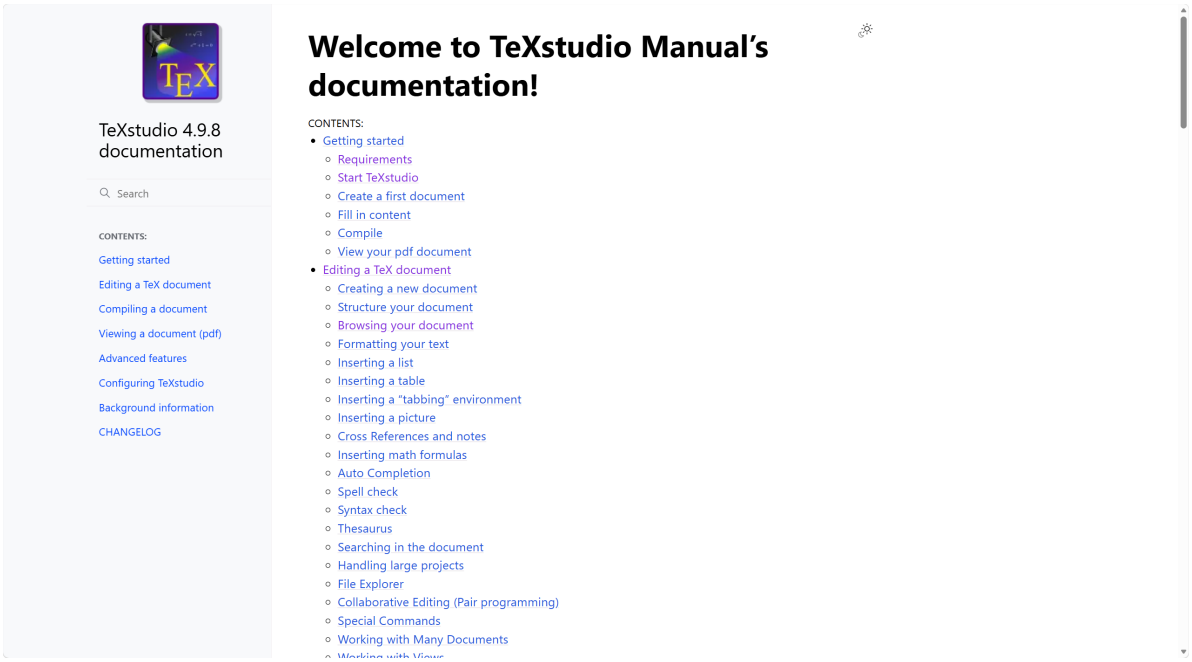


Figure 11: Landing page and table of contents of the TeXstudio manual used for the initial library.

Table 11: TeXstudio Verified tasks and information required beyond editor operation. The entries summarize the full task instructions supplied with ScienceBoard-VE.

Task	Task goal	Additional information or tool
A-01	Enter a molecular angle in the article.	Measurement in ChimeraX.
A-02	Enter a residue name in the article's table.	Residue information from ChimeraX.
A-03	Correct a DNA sequence in the article.	Sequence information from ChimeraX.
A-04	Correct a Fibonacci value in the article.	Computation in KAlgebra.
B-01	Complete a `\cite` command for an existing bibliography entry.	None.
B-02	Enter the COSPAR ID of Gemini 9A and add a citation.	Object information from Celestia.
B-03	Enter the number of provinces in Piemonte and add a citation.	Geographic information from GRASS GIS.
B-04	Add the BibTeX record for a specified paper to `refs.bib`.	Bibliographic search on Google Scholar.
C-01	Correct an atom count in the article.	Count obtained in ChimeraX.
C-02	Correct one letter in a λC deduction rule.	Knowledge of the deduction rule.
C-03	Repair an incomplete figure path.	The project's actual file path.
C-04	Change only the width of a specified figure.	None.
C-05	Correct a Lean 4 code fragment in the article.	Lean code knowledge; VS Code can be used for debugging.
C-06	Revise a formula until the document compiles.	TeX compilation feedback.
D-01	Read the document and answer the value of $BB(4)$.	Knowledge needed to resolve the question.
D-02	Read the document and answer the value of $B(5)$.	Knowledge needed to resolve the question.

Many tasks therefore have two components: obtaining the content to insert and editing or compiling the TeX document. TeXstudio documentation can support the latter, but it does not supply

measurements or object information from other scientific applications. A library constructed and evolved from TeXstudio documentation alone cannot cover all of these cross-application requirements, which helps explain its more limited improvement in Table 3.

O Research Setting and Baseline Reproduction Constraints of Resource2Skill

O.1 Research Setting and Knowledge Objectives

Resource2Skill and ScienceCUA both investigate how to transform human-created resources into reusable skills, but they differ in their research settings and knowledge objectives.

Resource2Skill primarily addresses authoring tasks across domains such as PPT, Web, Excel, Blender, CAD, UE5, and audio production. It distills skills from tutorial videos, repositories, articles, documentation, and reference artifacts. Its library provides retrievable and composable authoring techniques, code patterns, and visual references to help agents create new artifacts, with downstream evaluation primarily measuring artifact quality.

ScienceCUA aims to improve the ability of models and agents to operate scientific software. Starting from supplied PDF/HTML software documentation, it discovers supported operational tasks, derives task-level procedures, and organizes shared knowledge such as commands, syntax, and parameter constraints. These forms of knowledge are represented as Skills and References, respectively, to support agents in completing user tasks in deployed software environments. Downstream evaluation primarily measures task success.

Thus, although both approaches investigate resource-to-skill construction, they differ in how resources are used, how knowledge is organized, and what downstream evaluation emphasizes.

O.2 Limitations in Adapting the Released Implementation

At the implementation level, the relevant public interfaces of Resource2Skill do not directly provide the document-level knowledge-construction process required for our collections of PDF/HTML software documentation. Although the paper describes a construction operator that can generate multiple candidate skills from a single resource, the two interfaces directly relevant to our documentation inputs have the following limitations in the public code we inspected.

1. **`StaticArtifactConnector` does not directly support PDF manual parsing.** Its directory mode filters files using domain-specific extension lists, covering `.xlsx`, `.html/.htm`, `.pptx`, `.blend`, and selected audio and project formats, but excluding PDF. Consequently, the KAlgebra and Celestia PDF manuals used in our experiments cannot be directly processed through this mode. Its catalogue mode constructs candidates from supplied JSON fields, including descriptions and applicability; it does not itself parse the referenced manuals, discover knowledge units, or generate operational knowledge. StaticArtifactConnector implementation.

2. **`ArticleConnector` uses individual supplied pages as candidate-generation units.** It accepts a URL list, fetches each page, and produces at most one `CandidateSkill` per URL. Its optional model call primarily extracts a title, applicability, tags, and a short summary. The interface does not implement recursive documentation crawling, cross-page evidence aggregation, or the discovery of multiple procedural knowledge units within a page. Supplying a documentation homepage or a list of page URLs therefore does not directly provide a complete workflow for constructing operational knowledge from an entire documentation collection. ArticleConnector implementation.

Adapting these interfaces to our setting would require additional components for PDF parsing, HTML documentation discovery and collection, knowledge-unit decomposition, cross-page content

organization, and procedural-content generation, before packaging the results as `CandidateSkill` records. Resource2Skill's subsequent candidate processing and Skill Wiki construction remain reusable. However, the added components would directly determine the content, granularity, and coverage of the resulting library—properties central to the evaluation of resource-to-skill construction.

Such an adaptation would therefore extend beyond input-format conversion and introduce a knowledge-extraction method designed by us. Its downstream performance would make the respective contributions of Resource2Skill and the added extraction procedure difficult to distinguish. Given these differences in knowledge objectives and the adaptation requirements of the released implementation, we did not include Resource2Skill as an original-pipeline baseline alongside VisualSkill and SkillFoundry under the same evaluation setting.

P Supplementary Evaluation of Frontier Models

We additionally evaluate Claude Opus 5.5 and GPT-6 Astra on ScienceBoard-Verified, both with the `xhigh` reasoning setting. Claude Opus 5.5 shows comparatively limited computer-use performance in this scientific-software evaluation, so we do not further evaluate it with Skills and References in this supplementary study. GPT-6 Astra demonstrates substantially stronger performance. We therefore evaluate it with ScienceCUA's self-evolved Skills and References to examine whether a stronger base model can still benefit from external operational knowledge.

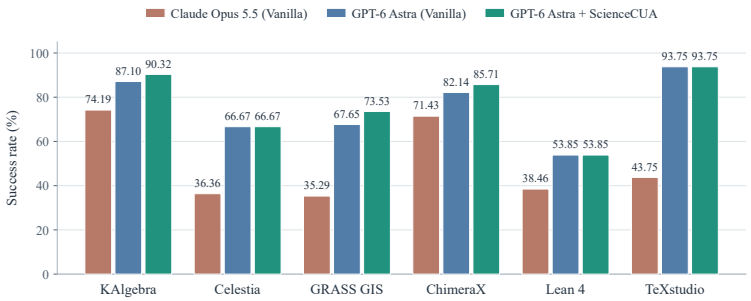


Figure 12: Success rates across the six applications in ScienceBoard-Verified. All models use the `xhigh` reasoning setting. ScienceCUA provides Skills and References that have undergone self-evolution. Labels above the bars give success rates in percent.

As shown in Figure 12, adding ScienceCUA's self-evolved library increases GPT-6 Astra's average success rate from 75.19% to 77.30%, a gain of 2.11 percentage points. These results show that the self-evolved library still provides a modest additional benefit on this stronger base model, supporting the discussion in Appendix A.